



ফ্লাটার শেখার ১৫ দিনের রোডম্যাপ

একদম শুরু থেকে অ্যাপ বানানো পর্যন্ত — সম্পূর্ণ বাংলা গাইড

নতুনদের জন্য

Dart + Flutter

প্রতিদিন ৩-৪ ঘণ্টা

কোড উদাহরণসহ

ধাপে ধাপে শেখার পরিকল্পনা · অনুশীলন · প্রজেক্ট · রেফারেন্স

শুরু করার আগে

ফ্লাটার (Flutter) কী?

Flutter হলো Google-এর তৈরি একটি ওপেন সোর্স **UI টুলকিট**। এর সবচেয়ে বড় সুবিধা হলো — আপনি **একবার কোড লিখে** সেই একই কোড থেকে Android, iOS, Web, Windows, macOS এবং Linux — ছয়টি প্ল্যাটফর্মের অ্যাপ বানাতে পারবেন। আলাদা করে Android-এর জন্য Java/Kotlin আর iOS-এর জন্য Swift শেখার দরকার নেই।

Flutter-এ অ্যাপ লিখতে হয় **Dart** নামের একটি প্রোগ্রামিং ল্যাঙ্গুয়েজ দিয়ে। Dart শেখা তুলনামূলক সহজ — আপনি যদি আগে C, C++, Java, JavaScript বা Python-এর সামান্য ধারণা রাখেন, তাহলে ২-৩ দিনেই Dart-এর মূল অংশ ধরে ফেলতে পারবেন। একেবারে নতুন হলেও চিন্তার কিছু নেই, এই রোডম্যাপে Dart একদম শূন্য থেকে শেখানো হয়েছে।

ফ্লাটারের মূল ধারণা: সবকিছুই Widget

Flutter-এ স্ক্রিনে যা কিছু দেখেন — বাটন, টেক্সট, ছবি, প্যাডিং, এমনকি পুরো স্ক্রিন — সবই এক একটি **Widget**। ছোট ছোট widget জোড়া দিয়ে বড় UI বানানো হয়, অনেকটা লেগো ব্লকের মতো। এই একটি ধারণা মাথায় গেঁথে গেলে Flutter-এর ৭০% বোঝা হয়ে যায়।

কেন ফ্লাটার শিখবেন?

- **এক কোডবেস, সব প্ল্যাটফর্ম** — সময় ও খরচ দুটোই অর্ধেক।
- **Hot Reload** — কোড পরিবর্তন করার ১ সেকেন্ডের মধ্যে ফোনে পরিবর্তন দেখা যায়। শেখার গতি অনেক বেড়ে যায়।
- **চাকরি ও ফ্রিল্যান্সিং** — বাংলাদেশ ও আন্তর্জাতিক মার্কেটে Flutter ডেভেলপারের চাহিদা দ্রুত বাড়ছে।
- **বিশাল কমিউনিটি** — pub.dev-এ ৪০ হাজারেরও বেশি রেডিমেড প্যাকেজ পাওয়া যায়।
- **পারফরম্যান্স** — Flutter সরাসরি নেটিভ কোডে কম্পাইল হয়, তাই অ্যাপ দ্রুত চলে।

এই রোডম্যাপ কীভাবে ব্যবহার করবেন

1. প্রতিদিনের জন্য **৩-৪ ঘণ্টা** সময় রাখুন। কম সময় দিলে ১৫ দিনের জায়গায় ২৫-৩০ দিন লাগবে — সমস্যা নেই, কিন্তু ধারাবাহিকতা ভাঙবেন না।

- প্রতিটি দিনে তিনটি অংশ আছে: **শেখা** (থিওরি) → **কোড** (উদাহরণ টাইপ করা) → **অনুশীলন** (নিজে বানানো)।
- কোড কপি-পেস্ট করবেন না, হাতে টাইপ করুন।** এটি শেখার গতি কয়েকগুণ বাড়িয়ে দেয়।
- প্রতিদিনের শেষে "চেকলিস্ট" মিলিয়ে দেখুন — সব টিক না পড়লে পরের দিনে যাবেন না।
- আটকে গেলে ২০ মিনিটের বেশি একা চেষ্টা করবেন না — এরর মেসেজটি Google-এ সার্চ করুন বা AI-কে জিজ্ঞেস করুন।

গুরুত্বপূর্ণ সতর্কতা

১৫ দিনে আপনি Flutter-এর **এক্সপার্ট** হবেন না — কেউই হয় না। কিন্তু ১৫ দিন পর আপনি নিজে থেকে একটি পূর্ণাঙ্গ, API-যুক্ত, একাধিক স্ক্রিনের অ্যাপ বানাতে পারবেন এবং যেকোনো Flutter কোড পড়ে বুঝতে পারবেন। এটাই এই রোডম্যাপের লক্ষ্য।

যা যা লাগবে

জিনিস	বিস্তারিত
কম্পিউটার	ন্যূনতম ৮ জিবি RAM (৪ জিবিতেও চলবে, তবে ইমুলেটরের বদলে আসল ফোন ব্যবহার করতে হবে)। ১০ জিবি ফাঁকা ডিস্ক স্পেস।
ইন্টারনেট	প্রথম দিনে ৫-৭ জিবি ডাউনলোড লাগবে (Flutter SDK + Android Studio)।
একটি অ্যান্ড্রয়েড ফোন	ঐচ্ছিক কিন্তু খুবই সুপারিশকৃত — ইমুলেটরের চেয়ে অনেক দ্রুত।
ধৈর্য	সবচেয়ে জরুরি উপাদান। প্রথম ৩ দিন কঠিন লাগবে, ৫ম দিন থেকে মজা লাগা শুরু হবে।

১৫ দিনের পূর্ণ পরিকল্পনা — এক নজরে

দিন	বিষয়	দিনশেষে যা পারবেন
১	পরিবেশ সেটআপ ও প্রথম অ্যাপ	ফোনে নিজের অ্যাপ চালানো
২	Dart বেসিক — ভ্যারিয়েবল, ফাংশন, কন্ডিশন, লুপ	ছোট প্রোগ্রাম লেখা
৩	Dart অ্যাডভান্স — OOP, List/Map, null safety	ক্লাস ও অবজেক্ট ব্যবহার
৪	Widget পরিচিতি — Text, Image, Icon, Button	প্রথম নিজের হাতে বানানো স্ক্রিন
৫	লেআউট — Row, Column, Container, Stack	যেকোনো ডিজাইন সাজানো
৬	StatefulWidget ও setState	বাটনে ক্লিক করলে স্ক্রিন বদলানো
৭	ListView, GridView, ScrollView	লম্বা লিস্টওয়ালা স্ক্রিন
৮	নেভিগেশন ও রাউটিং	একাধিক পেজের অ্যাপ
৯	ফর্ম, ইনপুট ও ভ্যালিডেশন	লগইন / রেজিস্ট্রেশন ফর্ম
১০	থিম, স্টাইল, ফন্ট, অ্যাসেট	সুন্দর ও প্রফেশনাল চেহারা
১১	API, HTTP ও JSON	ইন্টারনেট থেকে ডেটা আনা
১২	স্টেট ম্যানেজমেন্ট (Provider)	বড় অ্যাপের ডেটা সামলানো
১৩	লোকাল ডেটাবেজ ও স্টোরেজ	ডেটা ফোনে সেভ রাখা
১৪	Firebase, প্যাকেজ, পারমিশন, ক্যামেরা	রিয়েল-ওয়ার্ল্ড ফিচার
১৫	ফাইনাল প্রজেক্ট ও APK রিলিজ	নিজের সম্পূর্ণ অ্যাপ প্রকাশ

সোনাঁলি পরামর্শ

প্রতিদিন শেখার শেষে **১০ মিনিট** ব্যয় করুন আগের দিনের কোড আবার পড়তে। এই ছোট অভ্যাসটি ভুলে যাওয়ার হার ৬০% কমিয়ে দেয়।

দিন ০১

পরিবেশ সেটআপ ও প্রথম অ্যাপ চালানো

লক্ষ্য: দিনের শেষে আপনার নিজের ফোনে বা ইমুলেটরে আপনার লেখা অ্যাপ চলবে।

ধাপ ১: প্রয়োজনীয় সফটওয়্যার ডাউনলোড

1. **Flutter SDK** — flutter.dev/docs/get-started/install থেকে আপনার OS অনুযায়ী ZIP ফাইল নামান।
2. **Android Studio** — developer.android.com/studio থেকে। এটি ছাড়া Android অ্যাপ বানানো যাবে না (এতে Android SDK থাকে)।
3. **VS Code** — code.visualstudio.com। কোড লেখার জন্য এটি হালকা ও দ্রুত। (Android Studio-তেও লিখতে পারেন, ইচ্ছা আপনার।)
4. **Git** — git-scm.com থেকে।

ধাপ ২: Flutter ইনস্টল (Windows)

1. ZIP ফাইলটি `C:\src\flutter` ফোল্ডারে আনজিপ করুন। **সতর্কতা:** এমন কোনো ফোল্ডারে রাখবেন না যার পাথে স্পেস আছে (যেমন `C:\Program Files`) — এতে সমস্যা হয়।
2. Windows-এ সার্চ করুন "Environment Variables" → **Path** এডিট করুন → নতুন এন্ট্রি যোগ করুন:
`C:\src\flutter\bin`
3. নতুন একটি টার্মিনাল খুলে যাচাই করুন।

```
# ইনস্টল ঠিকভাবে হয়েছে কিনা দেখুন
flutter --version

# সবকিছু ঠিক আছে কিনা পুরো চেকআপ
flutter doctor
```

flutter doctor-এ লাল ক্রস দেখালে?

সবচেয়ে সাধারণ সমস্যা: **Android licenses not accepted**। সমাধান — টার্মিনালে চালান `flutter doctor --android-licenses` এবং প্রতিটি প্রশ্নে **y** চাপুন। আরেকটি সাধারণ সমস্যা: **cmdline-tools missing** → Android Studio খুলুন → Settings → Appearance & Behavior → System Settings → Android SDK → **SDK Tools** ট্যাব → **Android SDK Command-line Tools** টিক দিয়ে ইনস্টল করুন।

ধাপ ৩: ডিভাইস প্রস্তুত করা

বিকল্প ক — আসল ফোন (সুপারিশকৃত, দ্রুত)

1. ফোনের Settings → About phone → **Build number**-এ ৭ বার ট্যাপ করুন → Developer options চালু হবে।
2. Settings → Developer options → **USB debugging** চালু করুন।
3. USB কেবল দিয়ে কম্পিউটারে লাগান → ফোনে "Allow USB debugging?" এলে **Allow** দিন।
4. যাচাই করুন: `flutter devices`

বিকল্প খ — ইমুলেটর

Android Studio খুলুন → **Device Manager** → **Create Device** → Pixel 6 বেছে নিন → একটি System Image (যেমন API 34) ডাউনলোড করুন → Finish → ▶ বাটনে চাপ দিয়ে চালু করুন।

ধাপ ৪: প্রথম প্রজেক্ট তৈরি ও চালানো

```
# নতুন প্রজেক্ট বানান
flutter create my_first_app

# ফোল্ডারে ঢুকুন
cd my_first_app

# অ্যাপ চালান
flutter run
```

Hot Reload — ফ্লাটারের জাদু

অ্যাপ চলা অবস্থায় কোড বদলে সেভ করুন, তারপর টার্মিনালে **r** চাপুন → ১ সেকেন্ডে পরিবর্তন দেখা যাবে। **R** চাপলে Hot Restart (পুরো অ্যাপ রিস্টার্ট)। **q** চাপলে বন্ধ। VS Code-এ শুধু Ctrl+S দিলেই অটো হট রিলোড হয়।

ধাপ ৫: প্রজেক্ট ফোল্ডারে কী কী আছে?

ফোল্ডার / ফাইল	কাজ
lib/	আপনার সব Dart কোড এখানে থাকবে। ৯৫% সময় আপনি এই ফোল্ডারেই কাজ করবেন।
lib/main.dart	অ্যাপের শুরুর ফাইল। এখান থেকেই অ্যাপ চালু হয়।
pubspec.yaml	প্যাকেজ, ছবি, ফন্ট — সব ডিক্লেয়ার করার ফাইল। খুব গুরুত্বপূর্ণ।
android/, ios/	প্ল্যাটফর্ম-নির্দিষ্ট সেটিংস। পারমিশন বা অ্যাপ নাম বদলাতে মাঝে মাঝে এখানে আসতে হয়।
assets/	ছবি, ফন্ট, JSON ফাইল রাখার জায়গা (নিজে তৈরি করতে হয়)।
test/	অটোমেটেড টেস্ট লেখার জায়গা।

ধাপ ৬: প্রথম কোড লেখা

`lib/main.dart` ফাইলের সব মুছে দিয়ে নিচের কোডটি হাতে টাইপ করুন:

```
import 'package:flutter/material.dart';

// অ্যাপ শুরু হয় এই main() ফাংশন থেকে
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'আমার প্রথম অ্যাপ',
      debugShowCheckedModeBanner: false, // ডানপাশের DEBUG লেখা সরায়
      home: Scaffold(
        appBar: AppBar(
          title: const Text('আসসালামু আলাইকুম'),
          backgroundColor: Colors.teal,
        ),
        body: const Center(
          child: Text(
            'আমি ফ্লাটার শিখছি!',
            style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
          ),
        ),
      ),
    );
  }
}
```

এই কোডে কী হলো?

- `import` — Material Design-এর সব widget আনা হলো।
- `main()` — প্রোগ্রামের প্রবেশদ্বার। `runApp()` অ্যাপ চালু করে।
- `MaterialApp` — পুরো অ্যাপের মোড়ক; থিম, রাউট ইত্যাদি এখানে সেট হয়।
- `Scaffold` — একটি স্ক্রিনের কাঠামো: উপরে AppBar, মাঝে body, নিচে bottom bar।
- `Center` — ভিতরের widget-কে মাঝখানে বসায়।

আজকের অনুশীলন

১. AppBar-এর রং বদলে `Colors.deepPurple` করুন।
২. টেক্সটের সাইজ ৪০ করুন এবং রং লাল করুন।
৩. `Text` -এর জায়গায় `Icon(Icons.favorite, size: ৪০, color: Colors.pink)` বসিয়ে দেখুন।
৪. Scaffold-এ যোগ করুন: `backgroundColor: Colors.grey[২০০]`

দিন ১-এর চেকলিস্ট

- ☐ `flutter doctor` -এ কোনো লাল ক্রস নেই ☐ ফোন/ইমুলেটরে অ্যাপ চলেছে
- ☐ Hot Reload কাজ করেছে ☐ নিজে হাতে টেক্সট ও রং বদলাতে পেরেছি

দিন ০২

Dart বেসিক — ভ্যারিয়েবল, ফাংশন, কন্ডিশন, লুপ

লক্ষ্য: Dart-এর মৌলিক সিনট্যাক্স আয়ত্ত করা। আজ UI নয়, শুধু লজিক।

কোথায় প্র্যাকটিস করবেন?

ব্রাউজারে `dartpad.dev` খুলুন — কিছু ইনস্টল না করেই Dart কোড লিখে রান করতে পারবেন।
আজকের পুরো দিন DartPad-এ কাটান।

১. ভ্যারিয়েবল ও ডেটা টাইপ

```
void main() {  
  // স্পষ্টভাবে টাইপ লিখে  
  String name = 'রহিম';  
  int age = 22;  
  double height = 5.8;  
  bool isStudent = true;  
  
  // var – Dart নিজেই টাইপ বুঝে নেয়  
  var city = 'ঢাকা';           // String হিসেবে ধরা হবে  
  
  // final – একবার সেট হলে আর বদলানো যায় না  
  final country = 'বাংলাদেশ';  
  
  // const – কম্পাইল টাইমেই ফিক্সড (final এর চেয়েও কঠিন)  
  const double pi = 3.1416;  
  
  print('নাম: $name, বয়স: $age');           // String interpolation  
  print('আগামী বছর বয়স হবে: ${age + 1}'); // এক্সপ্রেশন হলে {} লাগে  
}
```

var, final, const — পার্থক্য মনে রাখার সহজ উপায়

var = পরে বদলানো যাবে। • **final** = রানটাইমে একবার সেট, আর বদলাবে না। • **const** = কোড লেখার সময়েই মান জানা, চিরকাল ফিক্সড।

নিয়ম: সন্দেহ হলে `final` ব্যবহার করুন। Flutter-এ যেখানে পারেন `const` দিন — এতে অ্যাপ দ্রুত চলে।

২. Null Safety — Dart-এর সবচেয়ে গুরুত্বপূর্ণ ফিচার

```
void main() {  
  String name = 'করিম';    // এটি কখনোই null হতে পারবে না  
  String? nickname;        // ? মানে – null হতে পারে  
  
  print(nickname);         // null প্রিন্ট হবে, এরর হবে না  
  
  // ?? – বাম দিকেরটা null হলে ডান দিকেরটা নাও  
  print(nickname ?? 'ডাকনাম নেই');  
  
  // ?. – null না হলে তবেই কাজটা করো  
  print(nickname?.length);  
  
  // ! – "আমি নিশ্চিত এটা null না" (সাবধানে ব্যবহার করুন!)  
  nickname = 'কর্ণ';  
  print(nickname!.length);  
}
```

নতুনদের #১ এরর

"Null check operator used on a null value" — এর মানে আপনি `!` ব্যবহার করেছেন এমন কিছুতে যেটি আসলে null ছিল। সমাধান: `!`-এর বদলে `??` বা `if (x != null)` ব্যবহার করুন।

৩. অপারেটর

```
void main() {  
    int a = 10, b = 3;  
  
    print(a + b);    // 13  
    print(a - b);    // 7  
    print(a * b);    // 30  
    print(a / b);    // 3.3333... (সবসময় double)  
    print(a ~/ b);    // 3 → ভাগফলের পূর্ণসংখ্যা অংশ  
    print(a % b);    // 1 → ভাগশেষ  
  
    // তুলনা  
    print(a > b);    // true  
    print(a == b);   // false  
  
    // লজিক্যাল  
    print(a > 5 && b < 5); // AND – দুটোই সত্য হতে হবে  
    print(a > 50 || b < 5); // OR – একটি সত্য হলেই চলবে  
    print(!(a > 5));      // NOT – উল্টো করে দেয়  
}
```

8. কন্ডিশন (if / else / switch)

```
void main() {  
    int marks = 78;  
  
    if (marks >= 80) {  
        print('A+ পেয়েছেন');  
    } else if (marks >= 70) {  
        print('A পেয়েছেন');  
    } else if (marks >= 60) {  
        print('A- পেয়েছেন');  
    } else {  
        print('আরও চেষ্টা করুন');  
    }  
  
    // টার্নারি - ছোট if-else এর শর্টকাট  
    String result = marks >= 33 ? 'পাস' : 'ফেল';  
    print(result);  
  
    // switch - অনেকগুলো নির্দিষ্ট মান মেলানোর জন্য  
    String day = 'শুক্রবার';  
    switch (day) {  
        case 'শুক্রবার':  
        case 'শনিবার':  
            print('ছুটির দিন');  
            break;  
        default:  
            print('কর্মদিবস');  
    }  
}
```

৫. লুপ

```
void main() {  
    // for লুপ – কতবার ঘুরবে জানা থাকলে  
    for (int i = 1; i <= 5; i++) {  
        print('গণনা: $i');  
    }  
  
    // for-in – লিস্টের প্রতিটি আইটেমের জন্য  
    List<String> fruits = ['আম', 'জাম', 'কাঁঠাল'];  
    for (String fruit in fruits) {  
        print('ফল: $fruit');  
    }  
  
    // forEach – আরও সংক্ষিপ্ত  
    fruits.forEach((f) => print('আমি খাই $f'));  
  
    // while – শর্ত সত্য থাকা পর্যন্ত চলবে  
    int count = 3;  
    while (count > 0) {  
        print('বাকি আছে $count');  
        count--;  
    }  
  
    // do-while – অন্তত একবার চলবেই  
    int n = 0;  
    do {  
        print('এটি অন্তত একবার চলবে');  
        n++;  
    } while (n < 1);  
  
    // break ও continue  
    for (int i = 1; i <= 10; i++) {  
        if (i == 3) continue; // ৩ বাদ দিয়ে পরেরটায় যাও  
        if (i == 6) break;    // ৬ এ পৌঁছালে লুপ থামাও  
        print(i);             // 1 2 4 5  
    }  
}
```

৬. ফাংশন

```
// সাধারণ ফাংশন
int add(int a, int b) {
  return a + b;
}

// আরো ফাংশন – এক লাইনের কাজ হলে
int multiply(int a, int b) => a * b;

// কিছু ফেরত না দিলে void
void greet(String name) {
  print('স্বাগতম, $name!');
}

// Optional named parameter – Flutter-এ সবচেয়ে বেশি ব্যবহৃত
void showUser({required String name, int age = 18, String? city}) {
  print('$name, $age, ${city ?? "অজানা"}');
}

// Optional positional parameter
String fullName(String first, [String? last]) {
  return last == null ? first : '$first $last';
}

void main() {
  print(add(5, 3)); // 8
  print(multiply(4, 2)); // 8
  greet('রহিম');
  showUser(name: 'করিম', city: 'সিলেট'); // নাম ধরে পাঠানো
  print(fullName('আব্দুল'));
}
```

named parameter কেন গুরুত্বপূর্ণ?

Flutter-এর প্রায় সব widget named parameter ব্যবহার করে — `Text('হাই', style: ..., textAlign: ...)` । তাই আজই এটি ভালো করে বুঝে নিন।

আজকের অনুশীলন (DartPad-এ)

১. একটি ফাংশন লিখুন যা একটি সংখ্যা জোড় না বিজোড় বলে দেয়।
২. ১ থেকে ১০০ পর্যন্ত সব জোড় সংখ্যার যোগফল বের করুন।
৩. একটি ফাংশন লিখুন যা নাম্বার নিয়ে গ্রেড (A+, A, B, F) রিটার্ন করে।
৪. নামের একটি লিস্ট বানিয়ে লুপ চালিয়ে প্রতিটির আগে "জনাব" যোগ করে প্রিন্ট করুন।
৫. একটি ফাংশন লিখুন যা ৫ এর নামতা প্রিন্ট করে।

দিন ২-এর চেকলিস্ট

- ☐ var/final/const-এর পার্থক্য জানি
- ☐ null safety (?, ??, !) বুঝি
- ☐ if-else ও লুপ নিজে লিখতে পারি
- ☐ named parameter দিয়ে ফাংশন লিখতে পারি

দিন ০৩

Dart অ্যাডভান্স — Collection, OOP ও Async

লক্ষ্য: ক্লাস-অবজেক্ট ও List/Map আয়ত্ত করা। এগুলো ছাড়া Flutter-এ এক পা-ও এগোনো যায় না।

১. List — একাধিক জিনিসের সারি

```
void main() {  
  List<String> names = ['রহিম', 'করিম', 'জসিম'];  
  
  print(names[0]);           // রহিম (গণনা শুরু ০ থেকে)  
  print(names.length);       // 3  
  print(names.first);         // রহিম  
  print(names.last);          // জসিম  
  print(names.isEmpty);       // false  
  
  names.add('সেলিম');         // শেষে যোগ  
  names.insert(0, 'আলিম');    // নির্দিষ্ট জায়গায়  
  names.remove('করিম');        // মান দিয়ে মুছুন  
  names.removeAt(0);           // ইনডেক্স দিয়ে মুছুন  
  
  // খুব দরকারি মেথড  
  var upper = names.map((n) => n.toUpperCase()).toList();  
  var filtered = names.where((n) => n.length > 4).toList();  
  bool has = names.contains('জসিম');  
  names.sort();  
  
  print(upper); print(filtered); print(has); print(names);  
}
```

২. Map — কী ও মানের জোড়া

```
void main() {
    Map<String, dynamic> student = {
        'name': 'রহিম',
        'age': 22,
        'isPassed': true,
    };

    print(student['name']);           // রহিম
    student['grade'] = 'A+';         // নতুন যোগ
    student.remove('age');           // মুছে ফেলা

    print(student.keys);             // সব কী
    print(student.values);           // সব মান
    print(student.containsKey('name'));

    // লুপ চালানো
    student.forEach((key, value) {
        print('$key → $value');
    });
}
```

Set — ডুপ্লিকেট ছাড়া সংগ্রহ

`Set<int> nums = {1, 2, 2, 3};` → ফলাফল `{1, 2, 3}`। একই জিনিস দুবার রাখতে না চাইলে Set ব্যবহার করুন।

৩. OOP — Class ও Object

```
class Student {
    // ফিল্ড / প্রপার্টি
    String name;
    int age;
    double? cgpa;

    // কনস্ট্রাক্টর
    Student(this.name, this.age, {this.cgpa});

    // নামযুক্ত কনস্ট্রাক্টর
    Student.guest() : name = 'অতিথি', age = 0;

    // মেথড
    void introduce() {
        print('আমি $name, বয়স $age বছর');
    }

    // getter – হিসাব করা প্রপার্টি
    String get status => (cgpa ?? 0) >= 3.5 ? 'ভালো' : 'সাধারণ';
}

void main() {
    var s1 = Student('রহিম', 22, cgpa: 3.8);
    s1.introduce();
    print(s1.status);

    var guest = Student.guest();
    guest.introduce();
}
```

8. Inheritance ও Override

```
class Animal {
  String name;
  Animal(this.name);

  void makeSound() {
    print('$name শব্দ করে');
  }
}

// Dog হলো Animal-এর সন্তান
class Dog extends Animal {
  Dog(String name) : super(name); // বাবার কনস্ট্রাক্টর ডাকা

  @override
  void makeSound() {
    print('$name ঘেউ ঘেউ করে'); // নিজের মতো বদলে নিলাম
  }

  void fetch() => print('$name বল আনে');
}

void main() {
  Dog d = Dog('টমি');
  d.makeSound(); // টমি ঘেউ ঘেউ করে
  d.fetch();
}
```

এটাই Flutter-এর ভিত্তি

আপনি যখন লেখেন `class MyApp extends StatelessWidget` — তখন আসলে এই inheritance-ই ব্যবহার করছেন। আর `@override Widget build(...)` মানে হলো StatelessWidget-এর build মেথডটি আপনি নিজের মতো লিখে নিলেন।

৫. Model Class — বাস্তবে যেভাবে ব্যবহার হয়

```
class Product {  
  final int id;  
  final String title;  
  final double price;  
  final String image;  
  
  Product({  
    required this.id,  
    required this.title,  
    required this.price,  
    required this.image,  
  });  
  
  // JSON (Map) থেকে অবজেক্ট বানানো – API-তে লাগবে  
  factory Product.fromJson(Map<String, dynamic> json) {  
    return Product(  
      id: json['id'],  
      title: json['title'],  
      price: (json['price'] as num).toDouble(),  
      image: json['image'] ?? '',  
    );  
  }  
  
  // অবজেক্ট থেকে JSON – সার্ভারে পাঠাতে লাগবে  
  Map<String, dynamic> toJson() => {  
    'id': id, 'title': title, 'price': price, 'image': image,  
  };  
}
```

৬. Async — অপেক্ষা করা কাজ

ইন্টারনেট থেকে ডেটা আনা বা ফাইল পড়া — এসব কাজে সময় লাগে। Dart এসব কাজ **ব্যাকগ্রাউন্ডে** করে, যাতে অ্যাপ আটকে না যায়। একে বলে asynchronous programming।

```
// Future = "ভবিষ্যতে একটা মান পাওয়া যাবে"
Future<String> fetchUserName() async {
  // ২ সেকেন্ড অপেক্ষা করার ভান করছি (যেন সার্ভার কল)
  await Future.delayed(Duration(seconds: 2));
  return 'রহিম উদ্দিন';
}

void main() async {
  print('ডেটা আনা শুরু...');

  String name = await fetchUserName(); // এখানে ২ সেকেন্ড অপেক্ষা
  print('পেলাম: $name');

  // এরর হ্যান্ডলিং
  try {
    String n = await fetchUserName();
    print(n);
  } catch (e) {
    print('সমস্যা হয়েছে: $e');
  } finally {
    print('কাজ শেষ');
  }
}
```

তিনটি নিয়ম মনে রাখুন

১. যে ফাংশনে `await` লিখবেন, সেই ফাংশনে **অবশ্যই** `async` লিখতে হবে।
২. `async` ফাংশন সবসময় `Future` রিটার্ন করে।
৩. `await` ভুলে গেলে আপনি ফলাফলের বদলে `Instance of 'Future<String>'` দেখবেন —
নতুনদের খুব সাধারণ ভুল।

আজকের অনুশীলন

১. `Book` নামে একটি ক্লাস বানান (`title`, `author`, `price`) এবং ৩টি বইয়ের লিস্ট বানিয়ে লুপে প্রিন্ট করুন।
২. সেই লিস্ট থেকে `where` দিয়ে ৫০০ টাকার বেশি দামের বই বের করুন।
৩. `Shape` ক্লাস বানিয়ে `Circle` ও `Square` দিয়ে `extend` করে `area()` `override` করুন।
৪. একটি `async` ফাংশন লিখুন যা ৩ সেকেন্ড পর "ডাউনলোড শেষ" প্রিন্ট করে।

দিন ৩-এর চেকলিস্ট

- ☐ List ও Map-এ কাজ করতে পারি
- ☐ ক্লাস ও কনস্ট্রাক্টর লিখতে পারি
- ☐ extends ও @override বুঝি
- ☐ async/await এর ধারণা পরিষ্কার

Widget পরিচিতি — Flutter UI-এর মূল ভিত্তি

লক্ষ্য: Widget কী, Stateless vs Stateful, এবং মৌলিক widget-গুলো ব্যবহার করা।

১. Widget Tree — গাছের মতো গঠন

Flutter-এ প্রতিটি স্ক্রিন একটি **গাছের** মতো — উপরে একটি মূল widget, তার ভিতরে ছোট ছোট widget, তার ভিতরে আরও ছোট। যেমন:

```
MaterialApp
├── Scaffold
│   ├── AppBar
│   │   └── Text('শিরোনাম')
│   └── Center
│       └── Column
│           ├── Text('হ্যালো')
│           ├── Icon(Icons.star)
│           └── ElevatedButton
│               └── Text('চাপুন')
```

২. দুই ধরনের Widget

StatelessWidget	StatefulWidget
যার চেহারা কখনো বদলায় না। যেমন: লোগো, শিরোনাম, আইকন।	যার চেহারা ব্যবহারকারীর কাজে বদলায়। যেমন: কাউন্টার, চেকবক্স, ফর্ম।
শুধু <code>build()</code> মেথড থাকে।	<code>build()</code> + <code>setState()</code> থাকে।
হালকা, দ্রুত। যেখানে পারেন এটি ব্যবহার করুন।	তুলনামূলক ভারী। দরকার হলেই কেবল ব্যবহার করুন।

শর্টকাট টাইপিং

VS Code-এ `stless` লিখে Enter চাপলে পুরো `StatelessWidget`-এর কাঠামো নিজে থেকে তৈরি হয়।
`stful` লিখলে `StatefulWidget` এই দুটো শর্টকাট আজই মুখস্থ করুন।

৩. Text — লেখা দেখানো

```
Text(  
  'ফ্লাটার শিখছি',  
  style: TextStyle(  
    fontSize: 22,  
    fontWeight: FontWeight.bold,      // w100 থেকে w900 পর্যন্তও দেওয়া যায়  
    color: Colors.deepPurple,  
    letterSpacing: 1.2,  
    height: 1.5,                      // লাইনের মধ্যে ফাঁকা  
    fontStyle: FontStyle.italic,  
    decoration: TextDecoration.underline,  
  ),  
  textAlign: TextAlign.center,  
  maxLines: 2,  
  overflow: TextOverflow.ellipsis,    // বেশি লম্বা হলে ... দেখাবে  
)
```

8. Container — সবচেয়ে বেশি ব্যবহৃত widget

```
Container(  
  width: 200,  
  height: 100,  
  padding: const EdgeInsets.all(16),           // ভিতরের ফাঁকা  
  margin: const EdgeInsets.symmetric(         // বাইরের ফাঁকা  
    horizontal: 20, vertical: 10,  
  ),  
  alignment: Alignment.center,  
  decoration: BoxDecoration(  
    color: Colors.teal,  
    borderRadius: BorderRadius.circular(16),  
    border: Border.all(color: Colors.black26, width: 2),  
    boxShadow: const [  
      BoxShadow(color: Colors.black26, blurRadius: 8, offset: Offset(0, 4)),  
    ],  
    gradient: const LinearGradient(  
      colors: [Colors.blue, Colors.purple],  
      begin: Alignment.topLeft,  
      end: Alignment.bottomRight,  
    ),  
  ),  
  child: const Text('আমি Container'),  
)
```

খুব সাধারণ এরর

"BoxDecoration painting cannot be used with color" — Container-এ একসাথে `color:` এবং `decoration:` দুটোই দেওয়া যায় না। `decoration` ব্যবহার করলে রংটিও `BoxDecoration` -এর ভিতরে দিন।

৫. Image — ছবি দেখানো

```
// ইন্টারনেট থেকে
Image.network(
  'https://picsum.photos/300',
  width: 200, height: 200,
  fit: BoxFit.cover,
  loadingBuilder: (c, child, progress) =>
    progress == null ? child : const CircularProgressIndicator(),
  errorBuilder: (c, e, s) => const Icon(Icons.broken_image, size: 60),
)

// অ্যাপের ভিতরের ফাইল থেকে (assets)
Image.asset('assets/images/logo.png', width: 150)

// গোল ছবি (প্রোফাইল পিকচার)
CircleAvatar(
  radius: 50,
  backgroundImage: NetworkImage('https://picsum.photos/200'),
)
```

asset ছবি যোগ করার ৩ ধাপ

১. প্রজেক্টের ভিতরে `assets/images/` ফোল্ডার বানান।
২. `pubspec.yaml` -এ লিখুন (ইনডেন্টেশন খুব গুরুত্বপূর্ণ — স্পেস গুনে দিন):

```
flutter:
  uses-material-design: true
  assets:
    - assets/images/
```

৩. টার্মিনালে `flutter pub get` চালিয়ে অ্যাপ রিস্টার্ট (Hot Reload যথেষ্ট নয়)।

৬. Icon ও Button

```
Icon(Icons.favorite, size: 40, color: Colors.red)

// প্রধান বাটন – ভরাট
ElevatedButton(
  onPressed: () { print('চাপা হয়েছে'); },
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.teal,
    foregroundColor: Colors.white,
    padding: const EdgeInsets.symmetric(horizontal: 30, vertical: 14),
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
  ),
  child: const Text('সাবমিট'),
)

// আইকনসহ বাটন
ElevatedButton.icon(
  onPressed: () {},
  icon: const Icon(Icons.download),
  label: const Text('ডাউনলোড'),
)

// বর্ডারওয়ালা বাটন
OutlinedButton(onPressed: () {}, child: const Text('বাতিল'))

// শুধু লেখা
TextButton(onPressed: () {}, child: const Text('আরও পড়ুন'))

// শুধু আইকন
IconButton(onPressed: () {}, icon: const Icon(Icons.delete))

// ভাসমান গোল বাটন (Scaffold-এ বসে)
FloatingActionButton(onPressed: () {}, child: const Icon(Icons.add))
```

বাটন নিষ্ক্রিয় করার নিয়ম

`onPressed: null` দিলে বাটনটি ধূসর ও নিষ্ক্রিয় হয়ে যায়। কাজ করাতে চাইলে অন্তত `onPressed: () {}` দিতেই হবে।

৭. Scaffold-এর পূর্ণ কাঠামো

```
Scaffold(  
  appBar: AppBar(  
    title: const Text('হোম'),  
    centerTitle: true,  
    actions: [ IconButton(onPressed: () {}, icon: const Icon(Icons.search)) ],  
  ),  
  drawer: const Drawer(child: Text('সাইড মেনু')), // বাম থেকে স্লাইড  
  body: const Center(child: Text('মূল অংশ')),  
  floatingActionButton: FloatingActionButton(  
    onPressed: () {}, child: const Icon(Icons.add),  
  ),  
  bottomNavigationBar: BottomNavigationBar(  
    items: const [  
      BottomNavigationBarItem(icon: Icon(Icons.home), label: 'হোম'),  
      BottomNavigationBarItem(icon: Icon(Icons.person), label: 'প্রোফাইল'),  
    ],  
  ),  
)
```

আজকের অনুশীলন — প্রোফাইল কার্ড

একটি স্ক্রিন বানান যেখানে থাকবে: গোল প্রোফাইল ছবি (CircleAvatar), নিচে নাম (বড় ও বোল্ড), তার নিচে পেশা (ধূসর রঙে), এবং সবার নিচে একটি "Follow" বাটন। পুরোটা একটি গোলাকার কোণা ও ছায়াযুক্ত Container-এর ভিতরে রাখুন।

দিন ০৫

লেআউট — Row, Column, Stack ও স্পেসিং

লক্ষ্য: যেকোনো ডিজাইন দেখে সেটি Flutter-এ সাজাতে পারা। এটিই সবচেয়ে গুরুত্বপূর্ণ দিন।

১. Column — উপর থেকে নিচে সাজানো

```
Column(  
  mainAxisAlignment: MainAxisAlignment.center, // উল্লম্ব বরাবর  
  crossAxisAlignment: CrossAxisAlignment.start, // আড়াআড়ি বরাবর  
  children: const [  
    Text('প্রথম লাইন'),  
    SizedBox(height: 12), // ফাঁকা জায়গা তৈরির সহজ উপায়  
    Text('দ্বিতীয় লাইন'),  
    Text('তৃতীয় লাইন'),  
  ],  
)
```

২. Row — বাম থেকে ডানে সাজানো

```
Row(  
  mainAxisAlignment: MainAxisAlignment.spaceBetween, // অনুভূমিক বরাবর  
  crossAxisAlignment: CrossAxisAlignment.center,  
  children: const [  
    Icon(Icons.person),  
    Text('রহিম'),  
    Icon(Icons.more_vert),  
  ],  
)
```

Main Axis ও Cross Axis — মনে রাখার সহজ নিয়ম

Column-এ: main axis = উল্লম্ব (↑), cross axis = অনুভূমিক (↔)।

Row-এ: main axis = অনুভূমিক (↔), cross axis = উল্লম্ব (↑)।

অর্থাৎ **main axis** সবসময় **widget** যেরদিকে সাজায় সেদিকে।

৩. Alignment-এর সব অপশন

MainAxisAlignment	যা করে
start	শুরুর দিকে চেপে বসে (ডিফল্ট)
end	শেষের দিকে
center	মাঝখানে
spaceBetween	প্রথম ও শেষ প্রান্তে, বাকি ফাঁকা সমান ভাগে মাঝে
spaceAround	প্রতিটির চারপাশে সমান ফাঁকা
spaceEvenly	সব ফাঁকা একদম সমান

CrossAxisAlignment	যা করে
start / end / center	আড়াআড়ি দিকে অবস্থান
stretch	পুরো প্রস্থ/উচ্চতা জুড়ে টেনে দেয়
baseline	লেখার নিচের রেখা মিলিয়ে দেয়

8. Expanded ও Flexible — জায়গা ভাগ করা

```
Row(  
  children: [  
    Expanded(  
      flex: 2, // ২ ভাগ জায়গা নেবে  
      child: Container(height: 80, color: Colors.red),  
    ),  
    Expanded(  
      flex: 1, // ১ ভাগ জায়গা নেবে  
      child: Container(height: 80, color: Colors.blue),  
    ),  
  ],  
)  
  
// Spacer – শুধু ফাঁকা জায়গা ঠেলে দেওয়ার জন্য  
Row(  
  children: const [  
    Text('বামে'),  
    Spacer(), // মাঝের সব জায়গা দখল করবে  
    Text('ডানে'),  
  ],  
)
```

সবচেয়ে ভয়ঙ্কর এরর: **RenderFlex overflowed by X pixels**

হলুদ-কালো ডোরাকাটা দাগ দেখলে বুঝবেন ভিতরের জিনিস জায়গার চেয়ে বড় হয়ে গেছে।

সমাধান: ১) Row-এর ভিতরের widget-কে `Expanded` দিয়ে মুড়ে দিন। ২) Column খুব লম্বা হলে পুরোটা `SingleChildScrollView` -এ মুড়ে দিন। ৩) লম্বা টেক্সটে `overflow: TextOverflow.ellipsis` দিন।

৫. Stack — একটির উপর আরেকটি

```
Stack(  
  children: [  
    Image.network('https://picsum.photos/400/250', fit: BoxFit.cover),  
  
    // ছবির উপরে লেখা বসানো  
    Positioned(  
      bottom: 16, left: 16,  
      child: Container(  
        padding: const EdgeInsets.all(8),  
        color: Colors.black54,  
        child: const Text('সুন্দরবন', style: TextStyle(color: Colors.white)),  
      ),  
    ),  
  
    // ডান-উপরে ব্যাজ  
    const Positioned(  
      top: 8, right: 8,  
      child: CircleAvatar(radius: 14, child: Text('৫')),  
    ),  
  ],  
)
```

৬. Padding, SizedBox ও Card

```
Padding(  
  padding: const EdgeInsets.all(16),  
  child: const Text('চারদিকে ফাঁকা'),  
)  
  
// EdgeInsets-এর সব ধরন  
EdgeInsets.all(16)  
EdgeInsets.symmetric(horizontal: 20, vertical: 10)  
EdgeInsets.only(left: 8, top: 16, right: 8, bottom: 0)  
EdgeInsets.fromLTRB(8, 16, 8, 0) // left, top, right, bottom  
  
// Card – রেডিমেড ছায়াযুক্ত বাক্স  
Card(  
  elevation: 4,  
  margin: const EdgeInsets.all(12),  
  shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(14)),  
  child: const ListTile(  
    leading: Icon(Icons.notifications),  
    title: Text('নতুন বার্তা'),  
    subtitle: Text('আপনার একটি নতুন বার্তা এসেছে'),  
    trailing: Icon(Icons.arrow_forward_ios, size: 16),  
  ),  
)
```

৭. পূর্ণ উদাহরণ — একটি প্রোডাক্ট কার্ড

```
Card(  
  elevation: 3,  
  shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),  
  child: Column(  
    crossAxisAlignment: CrossAxisAlignment.start,  
    children: [  
      ClipRRect(  
        borderRadius: const BorderRadius.vertical(top: Radius.circular(16)),  
        child: Image.network('https://picsum.photos/400/200',  
          height: 140, width: double.infinity, fit: BoxFit.cover),  
      ),  
      Padding(  
        padding: const EdgeInsets.all(12),  
        child: Column(  
          crossAxisAlignment: CrossAxisAlignment.start,  
          children: [  
            const Text('স্মার্ট ঘড়ি',  
              style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),  
            const SizedBox(height: 4),  
            Text('ওয়াটারপ্রুফ, ৭ দিন ব্যাটারি',  
              style: TextStyle(color: Colors.grey[600], fontSize: 13)),  
            const SizedBox(height: 10),  
            Row(  
              mainAxisAlignment: MainAxisAlignment.spaceBetween,  
              children: [  
                const Text('ট ২,৪৯৯',  
                  style: TextStyle(fontSize: 17, color: Colors.teal,  
                    fontWeight: FontWeight.bold)),  
                ElevatedButton(onPressed: () {}, child: const Text('কিনুন')),  
              ],  
            ),  
          ],  
        ),  
      ],  
    ),  
  ),  
)
```

আজকের অনুশীলন

১. উপরের প্রোডাক্ট কার্ডটি নিজে টাইপ করে বানান।
২. একটি ক্যালকুলেটরের বাটন লেআউট বানান (৪×৪ গ্রিড, Row ও Column দিয়ে)।
৩. Stack ব্যবহার করে একটি ছবির উপর "SALE 50%" লাল ব্যাজ বসান।
৪. একটি স্ক্রিনে ৩টি কার্ড পাশাপাশি বসান যেখানে মাঝেরটি দ্বিগুণ চওড়া (Expanded flex ব্যবহার করে)।

দিন ০৬

StatefulWidget ও setState — অ্যাপকে জীবন্ত করা

লক্ষ্য: ব্যবহারকারীর ক্লিকে স্ক্রিন পরিবর্তন করতে শেখা।

১. State কী?

State মানে হলো এমন তথ্য যা **সময়ের সাথে বদলায়** এবং বদলালে স্ক্রিনও বদলাতে হয়। যেমন — কাউন্টারের সংখ্যা, চেকবক্স টিক আছে কি নেই, লোডিং চলছে কিনা, লিস্টে কয়টি আইটেম আছে।

২. ক্লাসিক কাউন্টার উদাহরণ

```
import 'package:flutter/material.dart';

class CounterPage extends StatefulWidget {
  const CounterPage({super.key});

  @override
  State<CounterPage> createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int _count = 0; // এটিই আমাদের state

  void _increment() {
    setState(() { // setState = "স্ক্রিন আবার আঁকো"
      _count++;
    });
  }

  void _decrement() {
    setState(() {
      if (_count > 0) _count--;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('কাউন্টার')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text('আপনি চেপেছেন', style: TextStyle(fontSize: 18)),
            Text('$_count',
              style: const TextStyle(fontSize: 64, fontWeight: FontWeight.bold)),
            const SizedBox(height: 20),
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                ElevatedButton(onPressed: _decrement, child: const Text(' - ')),
                const SizedBox(width: 20),
                ElevatedButton(onPressed: _increment, child: const Text(' + ')),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

```
    ),  
  );  
}  
}
```

setState ছাড়া মান বদলালে কী হয়?

ভেরিয়েবলের মান ঠিকই বদলাবে, কিন্তু স্ক্রিনে কিছুই দেখা যাবে না — কারণ Flutter জানে না যে আবার আঁকতে হবে। নতুনদের এটি সবচেয়ে বড় বিভ্রান্তি। মনে রাখুন: **মান বদলালেই setState।**

৩. Widget Lifecycle — জীবনচক্র

```
class _MyPageState extends State<MyPage> {  
  
  @override  
  void initState() {  
    super.initState();  
    // widget প্রথমবার তৈরি হওয়ার সময় ঠিক একবার চলে।  
    // API কল, controller তৈরি – এসব এখানে করুন।  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    // প্রতিবার setState হলেই আবার চলে। এখানে ভারী কাজ করবেন না!  
    return const SizedBox();  
  }  
  
  @override  
  void dispose() {  
    // widget বন্ধ হওয়ার সময় চলে। controller বন্ধ করুন এখানে।  
    super.dispose();  
  }  
}
```

৪. শর্ত অনুযায়ী UI দেখানো

```
bool _isLoading = false;
bool _isLoggedIn = true;

// পদ্ধতি ১ – টার্নারি
_isLoading
  ? const CircularProgressIndicator()
  : const Text('ডেটা লোড হয়েছে')

// পদ্ধতি ২ – children লিস্টের ভিতরে if
Column(
  children: [
    const Text('সবসময় দেখাবে'),
    if (_isLoggedIn) const Text('স্বাগতম!'),
    if (!_isLoggedIn) const Text('অনুগ্রহ করে লগইন করুন'),
  ],
)

// পদ্ধতি ৩ – লুকানো কিন্তু জায়গা রেখে দেওয়া
Visibility(
  visible: _isLoggedIn,
  child: const Text('গোপন তথ্য'),
)
```

৫. ব্যবহারিক উদাহরণ — টুডু লিস্ট

```
class TodoPage extends StatefulWidget {
  const TodoPage({super.key});
  @override
  State<TodoPage> createState() => _TodoPageState();
}

class _TodoPageState extends State<TodoPage> {
  final List<String> _tasks = [];
  final TextEditingController _ctrl = TextEditingController();

  void _addTask() {
    if (_ctrl.text.trim().isEmpty) return;
    setState(() {
      _tasks.add(_ctrl.text.trim());
      _ctrl.clear();
    });
  }

  void _removeTask(int index) {
    setState(() => _tasks.removeAt(index));
  }

  @override
  void dispose() {
    _ctrl.dispose(); // মেমোরি লিক এড়াতে অবশ্যই লাগবে
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('আমার কাজের তালিকা')),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(12),
            child: Row(
              children: [
                Expanded(
                  child: TextField(
                    controller: _ctrl,
                    decoration: const InputDecoration(
                      hintText: 'নতুন কাজ লিখুন',
                      border: OutlineInputBorder(),
                    ),
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

```

    ),
    const SizedBox(width: 8),
    ElevatedButton(onPressed: _addTask, child: const Text('যোগ')),
  ],
),
),
Expanded(
  child: _tasks.isEmpty
    ? const Center(child: Text('কোনো কাজ নেই'))
    : ListView.builder(
        itemCount: _tasks.length,
        itemBuilder: (context, i) => ListTile(
          leading: CircleAvatar(child: Text('${i + 1}')),
          title: Text(_tasks[i]),
          trailing: IconButton(
            icon: const Icon(Icons.delete, color: Colors.red),
            onPressed: () => _removeTask(i),
          ),
        ),
      ),
),
),
],
),
);
}
}

```

আজকের অনুশীলন

১. উপরের টুডু অ্যাপটি সম্পূর্ণ নিজে টাইপ করে চালান।
২. প্রতিটি কাজের পাশে একটি Checkbox যোগ করুন যা টিক দিলে লেখাটি কাটা (lineThrough) দেখাবে।
৩. একটি "ডার্ক মোড" সুইচ বানান যা চাপলে Scaffold-এর background কালো হয়ে যায়।
৪. একটি লাইক বাটন বানান — চাপলে আইকন ফাঁকা হার্ট থেকে লাল ভরাট হার্ট হবে এবং সংখ্যা বাড়বে।

দিন ০৭

ListView, GridView ও স্ক্রলিং

লক্ষ্য: অনেক ডেটা সুন্দরভাবে লিস্ট বা গ্রিড আকারে দেখানো।

১. ListView-এর তিন ধরন

```
// ১) ListView – অল্প কয়েকটি আইটেম, হাতে লেখা
ListView(
  padding: const EdgeInsets.all(8),
  children: const [
    ListTile(title: Text('প্রথম')),
    ListTile(title: Text('দ্বিতীয়')),
  ],
)

// ২) ListView.builder – ★ সবচেয়ে বেশি ব্যবহৃত ★
// শুধু স্ক্রিনে দৃশ্যমান আইটেমগুলোই তৈরি করে, তাই হাজারো আইটেমেও দ্রুত চলে
ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, index) {
    return ListTile(
      leading: CircleAvatar(child: Text('${index + 1}')),
      title: Text(items[index]),
    );
  },
)

// ৩) ListView.separated – আইটেমের মাঝে বিভাজক রেখা
ListView.separated(
  itemCount: items.length,
  separatorBuilder: (context, i) => const Divider(height: 1),
  itemBuilder: (context, i) => ListTile(title: Text(items[i])),
)
```

কখন কোনটি?

আইটেম সংখ্যা ১০-এর কম ও নির্দিষ্ট হলে `ListView` | ডেটা লিস্ট থেকে আসলে বা সংখ্যা অনিশ্চিত হলে সবসময় `ListView.builder` | মাঝে লাইন চাইলে `.separated` |

২. GridView — গ্রিড আকারে

```
GridView.builder(  
  padding: const EdgeInsets.all(12),  
  gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(  
    crossAxisCount: 2,      // এক সারিতে কয়টি  
    crossAxisSpacing: 12,   // পাশাপাশি ফাঁকা  
    mainAxisSpacing: 12,    // উপর-নিচ ফাঁকা  
    childAspectRatio: 0.75, // প্রস্থ ÷ উচ্চতা  
  ),  
  itemCount: products.length,  
  itemBuilder: (context, i) => Card(  
    child: Column(  
      children: [  
        Expanded(child: Image.network(products[i].image, fit: BoxFit.cover)),  
        Padding(  
          padding: const EdgeInsets.all(8),  
          child: Text(products[i].title, maxLines: 1,  
            overflow: TextOverflow.ellipsis),  
        ),  
      ],  
    ),  
  ),  
)
```

৩. স্ক্রলিং সংক্রান্ত সমস্যা ও সমাধান

```
// লম্বা Column overflow করলে – পুরোটা মুড়ে দিন
SingleChildScrollView(
  child: Column(children: [ /* অনেক widget */ ]),
)

// Column-এর ভিতরে ListView রাখলে "unbounded height" এরর হয়।
// সমাধান ১ – Expanded দিয়ে মুড়ুন
Column(
  children: [
    const Text('শিরোনাম'),
    Expanded(child: ListView.builder(...)),
  ],
)

// সমাধান ২ – ListView-কে নিজে স্ক্রল করতে না দিয়ে
ListView.builder(
  shrinkWrap: true,
  physics: const NeverScrollableScrollPhysics(),
  itemCount: 5,
  itemBuilder: (c, i) => Text('আইটেম $i'),
)
```

এরর: "Vertical viewport was given unbounded height"

এর মানে — আপনি একটি স্ক্রলযোগ্য widget (ListView) আরেকটি স্ক্রলযোগ্য বা অসীম উচ্চতার widget (Column/SingleChildScrollView)-এর ভিতরে রেখেছেন। উপরের দুটি সমাধানের যেকোনো একটি প্রয়োগ করুন।

8. Pull-to-Refresh ও ডিলিট করার জন্য সোয়াইপ

```
// নিচে টেনে রিফ্রেশ
RefreshIndicator(
  onRefresh: () async {
    await Future.delayed(const Duration(seconds: 1));
    setState(() { /* ডেটা আবার লোড */ });
  },
  child: ListView.builder(...),
)

// পাশে সোয়াইপ করে মুছে ফেলা
Dismissible(
  key: ValueKey(items[i]),
  direction: DismissDirection.endToStart,
  background: Container(
    color: Colors.red,
    alignment: Alignment.centerRight,
    padding: const EdgeInsets.only(right: 20),
    child: const Icon(Icons.delete, color: Colors.white),
  ),
  onDismissed: (_) => setState(() => items.removeAt(i)),
  child: ListTile(title: Text(items[i])),
)
```

৫. মডেল ব্যবহার করে সম্পূর্ণ লিস্ট

```
class Contact {  
  final String name, phone;  
  Contact(this.name, this.phone);  
}  
  
final contacts = [  
  Contact('রহিম উদ্দিন', '০১৭১১১১১১১'),  
  Contact('করিম মিয়া', '০১৮২২২২২২২২'),  
  Contact('সালমা খাতুন', '০১৯৩৩৩৩৩৩৩৩'),  
];  
  
// ব্যবহার  
ListView.separated(  
  itemCount: contacts.length,  
  separatorBuilder: (_, __) => const Divider(height: 1),  
  itemBuilder: (context, i) {  
    final c = contacts[i];  
    return ListTile(  
      leading: CircleAvatar(child: Text(c.name[0])),  
      title: Text(c.name),  
      subtitle: Text(c.phone),  
      trailing: IconButton(  
        icon: const Icon(Icons.call, color: Colors.green),  
        onPressed: () {},  
      ),  
      onTap: () {},  
    );  
  },  
)
```

আজকের অনুশীলন

১. ২০টি ফোন নম্বরের কন্টাক্ট লিস্ট বানান (উপরের মডেল ব্যবহার করে)।
২. একটি ২-কলামের প্রোডাক্ট গ্রিড বানান যেখানে ছবি, নাম ও দাম থাকবে।
৩. লিস্ট Dismissible যোগ করে সোয়াইপে ডিলিট চালু করুন।
৪. উপরে একটি সার্চ বক্স বসিয়ে `where()` দিয়ে লিস্ট ফিল্টার করুন।

দিন ০৮

নেভিগেশন ও রাউটিং — একাধিক পেজের অ্যাপ

লক্ষ্য: এক স্ক্রিন থেকে আরেক স্ক্রিনে যাওয়া এবং ডেটা আদান-প্রদান।

১. সহজ নেভিগেশন (push / pop)

```
// দ্বিতীয় পেজে যাওয়া
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => const SecondPage()),
);

// আগের পেজে ফিরে আসা
Navigator.pop(context);

// বর্তমান পেজ বদলে নতুন পেজ (ফিরে আসা যাবে না)
Navigator.pushReplacement(
  context,
  MaterialPageRoute(builder: (_) => const HomePage()),
);

// সব আগের পেজ মুছে নতুন পেজ (লগআউটের সময় দরকারি)
Navigator.pushAndRemoveUntil(
  context,
  MaterialPageRoute(builder: (_) => const LoginPage()),
  (route) => false,
);
```

২. ডেটাসহ পেজে যাওয়া

```
// পাঠানোর পেজ
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => DetailPage(title: 'স্মার্ট ঘড়ি', price: 2499),
  ),
);

// গ্রহণকারী পেজ
class DetailPage extends StatelessWidget {
  final String title;
  final double price;

  const DetailPage({super.key, required this.title, required this.price});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(title)),
      body: Center(child: Text('দাম: ট $price')),
    );
  }
}
```

৩. পেজ থেকে ফলাফল ফেরত আনা

```
// প্রথম পেজে
final result = await Navigator.push(
  context,
  MaterialPageRoute(builder: (_) => const PickCityPage()),
);
if (result != null) {
  setState(() => selectedCity = result);
}

// দ্বিতীয় পেজে – ফেরত পাঠানো
Navigator.pop(context, 'চট্টগ্রাম');
```

8. Named Routes — বড় অ্যাপের জন্য পরিচ্ছন্ন উপায়

```
MaterialApp(  
  initialRoute: '/',  
  routes: {  
    '/': (context) => const HomePage(),  
    '/profile': (context) => const ProfilePage(),  
    '/settings': (context) => const SettingsPage(),  
  },  
)  
  
// ব্যবহার  
Navigator.pushNamed(context, '/profile');  
  
// আর্গুমেন্টসহ  
Navigator.pushNamed(context, '/detail', arguments: {'id': 5});  
  
// গ্রহণকারী পেজে  
final args = ModalRoute.of(context)!.settings.arguments as Map;  
print(args['id']);
```

৫. ট্যাব নেভিগেশন (BottomNavigationBar)

```
class MainPage extends StatefulWidget {
  const MainPage({super.key});
  @override
  State<MainPage> createState() => _MainPageState();
}

class _MainPageState extends State<MainPage> {
  int _index = 0;

  final List<Widget> _pages = const [
    HomePage(), SearchPage(), ProfilePage(),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: _pages[_index],
      bottomNavigationBar: BottomNavigationBar(
        currentIndex: _index,
        onTap: (i) => setState(() => _index = i),
        selectedItemColor: Colors.teal,
        type: BottomNavigationBarType.fixed,
        items: const [
          BottomNavigationBarItem(icon: Icon(Icons.home), label: 'হোম'),
          BottomNavigationBarItem(icon: Icon(Icons.search), label: 'খুঁজুন'),
          BottomNavigationBarItem(icon: Icon(Icons.person), label: 'প্রোফাইল'),
        ],
      ),
    );
  }
}
```

৬. Drawer — সাইড মেনু

```
Scaffold(  
  appBar: AppBar(title: const Text('হোম')),  
  drawer: Drawer(  
    child: ListView(  
      padding: EdgeInsets.zero,  
      children: [  
        const UserAccountsDrawerHeader(  
          accountName: Text('রহিম উদ্দিন'),  
          accountEmail: Text('rahim@example.com'),  
          currentAccountPicture: CircleAvatar(child: Text('র')),  
        ),  
        ListTile(  
          leading: const Icon(Icons.home),  
          title: const Text('হোম'),  
          onTap: () => Navigator.pop(context),  
        ),  
        ListTile(  
          leading: const Icon(Icons.settings),  
          title: const Text('সেটিংস'),  
          onTap: () {  
            Navigator.pop(context); // আগে drawer বন্ধ করুন  
            Navigator.pushNamed(context, '/settings');  
          },  
        ),  
      ],  
    ),  
  ),  
  body: const Center(child: Text('মূল পাতা')),  
)
```

৭. Dialog, SnackBar ও BottomSheet

```
// নিচে ছোট বার্তা
ScaffoldMessenger.of(context).showSnackBar(
  const SnackBar(content: Text('সফলভাবে সেভ হয়েছে')),
);

// নিশ্চিতকরণ ডায়ালগ
showDialog(
  context: context,
  builder: (ctx) => AlertDialog(
    title: const Text('নিশ্চিত?'),
    content: const Text('আপনি কি সত্যিই মুছে ফেলতে চান?'),
    actions: [
      TextButton(onPressed: () => Navigator.pop(ctx), child: const Text('না')),
      TextButton(
        onPressed: () { Navigator.pop(ctx); /* মুছুন */ },
        child: const Text('হ্যাঁ', style: TextStyle(color: Colors.red)),
      ),
    ],
  ),
);

// নিচ থেকে উঠে আসা শিট
showModalBottomSheet(
  context: context,
  builder: (_) => SizedBox(
    height: 200,
    child: Column(children: const [ListTile(title: Text('শেয়ার'))]),
  ),
);
```

আজকের অনুশীলন

৩টি ট্যাবওয়ালা (হোম, ক্যাটাগরি, প্রোফাইল) একটি অ্যাপ বানান। হোম ট্যাবে একটি প্রোডাক্ট লিস্ট থাকবে, কোনো আইটেমে ট্যাপ করলে বিস্তারিত পেজে যাবে (ডেটাসহ)। একটি Drawer যোগ করুন এবং ডিলিট বাটনে AlertDialog দেখান।

দিন ০৯

ফর্ম, ইনপুট ও ভ্যালিডেশন

লক্ষ্য: লগইন ও রেজিস্ট্রেশন ফর্ম বানানো এবং ইনপুট যাচাই করা।

১. TextField — মৌলিক ইনপুট

```
final TextEditingController emailCtrl = TextEditingController();

TextField(
  controller: emailCtrl,
  keyboardType: TextInputType.emailAddress,
  decoration: InputDecoration(
    labelText: 'ইমেইল',
    hintText: 'example@mail.com',
    prefixIcon: const Icon(Icons.email),
    border: OutlineInputBorder(borderRadius: BorderRadius.circular(10)),
    filled: true,
    fillColor: Colors.grey[100],
  ),
)

// পাসওয়ার্ড ফিল্ড
TextField(
  obscureText: true,
  decoration: const InputDecoration(
    labelText: 'পাসওয়ার্ড',
    prefixIcon: Icon(Icons.lock),
    border: OutlineInputBorder(),
  ),
)

// মান পড়া
print(emailCtrl.text);
```

keyboardType-এর দরকারি মানগুলো

text · number · phone · emailAddress · multiline · datetime · url | সঠিকটি দিলে
ব্যবহারকারীর অভিজ্ঞতা অনেক ভালো হয়।


```

        return 'ইমেইল দিন';
      }
      if (!value.contains('@')) {
        return 'সঠিক ইমেইল দিন';
      }
      return null; // null মানে কোনো ভুল নেই
    },
  ),
  const SizedBox(height: 16),
  TextFormField(
    controller: _passCtrl,
    obscureText: _obscure,
    decoration: InputDecoration(
      labelText: 'পাসওয়ার্ড',
      border: const OutlineInputBorder(),
      suffixIcon: IconButton(
        icon: Icon(_obscure ? Icons.visibility : Icons.visibility_off),
        onPressed: () => setState(() => _obscure = !_obscure),
      ),
    ),
    validator: (v) =>
      (v == null || v.length < 6) ? 'কমপক্ষে ৬ অক্ষর দিন' : null,
  ),
  const SizedBox(height: 24),
  SizedBox(
    width: double.infinity,
    height: 48,
    child: ElevatedButton(
      onPressed: _submit,
      child: const Text('লগইন করুন'),
    ),
  ),
],
),
),
),
);
}
}

```

Form কীভাবে কাজ করে — ৩টি অংশ

১. `GlobalKey<FormState>` — ফর্মের নিয়ন্ত্রক।
২. প্রতিটি `TextFormField` -এ `validator` — ভুল হলে বার্তা, ঠিক হলে `null` রিটার্ন।
৩. `_formKey.currentState!.validate()` — একসাথে সব ফিল্ড যাচাই করে true/false দেয়।

৩. অন্যান্য ইনপুট widget

```
// চেকবক্স
Checkbox(value: _agreed, onChanged: (v) => setState(() => _agreed = v!))

// সুইচ
Switch(value: _isDark, onChanged: (v) => setState(() => _isDark = v))

// রেডিও
Radio<String>(
  value: 'পুরুষ', groupValue: _gender,
  onChanged: (v) => setState(() => _gender = v!),
)

// ড্রপডাউন
DropdownButtonFormField<String>(
  value: _city,
  decoration: const InputDecoration(labelText: 'শহর', border: OutlineInputBorder()),
  items: const [
    DropdownMenuItem(value: 'ঢাকা', child: Text('ঢাকা')),
    DropdownMenuItem(value: 'চট্টগ্রাম', child: Text('চট্টগ্রাম')),
    DropdownMenuItem(value: 'সিলেট', child: Text('সিলেট')),
  ],
  onChanged: (v) => setState(() => _city = v),
)

// স্লাইডার
Slider(value: _volume, min: 0, max: 100,
  onChanged: (v) => setState(() => _volume = v))

// তারিখ বাছাই
final date = await showDatePicker(
  context: context,
  initialDate: DateTime.now(),
  firstDate: DateTime(1950),
  lastDate: DateTime(2100),
);
```

৪. কিবোর্ড ও ফোকাস সামলানো

```
// কিবোর্ড খুললে যাতে overflow না হয়
Scaffold(
  resizeToAvoidBottomInset: true,
  body: SingleChildScrollView(child: Form(...)),
)

// বাইরে ট্যাপ করলে কিবোর্ড বন্ধ
GestureDetector(
  onTap: () => FocusScope.of(context).unfocus(),
  child: Scaffold(...),
)

// Enter চাপলে পরের ফিল্ডে যাওয়া
TextFormField(
  textInputAction: TextInputAction.next,
  onFieldSubmitted: (_) => FocusScope.of(context).nextFocus(),
)
```

আজকের অনুশীলন — রেজিস্ট্রেশন ফর্ম

একটি সম্পূর্ণ রেজিস্ট্রেশন ফর্ম বানান: নাম, ইমেইল, ফোন (১১ ডিজিট যাচাই), পাসওয়ার্ড, কনফার্ম পাসওয়ার্ড (দুটি মিলছে কিনা যাচাই), জেন্ডার (Radio), শহর (Dropdown), জন্মতারিখ (DatePicker), এবং "শর্তাবলি মানছি" (Checkbox — টিক না দিলে সাবমিট হবে না)। সব ঠিক থাকলে Snackbar-এ সব তথ্য দেখান।

দিন ১০

থিম, স্টাইল, ফন্ট ও অ্যানিমেশন

লক্ষ্য: অ্যাপকে দেখতে প্রফেশনাল ও সুন্দর করা।

১. অ্যাপ থিম — একবার সেট, সব জায়গায় কাজ

```
MaterialApp(  
  theme: ThemeData(  
    useMaterial3: true,  
    colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),  
  
    appBarTheme: const AppBarTheme(  
      backgroundColor: Colors.teal,  
      foregroundColor: Colors.white,  
      centerTitle: true,  
      elevation: 0,  
    ),  
  
    elevatedButtonTheme: ElevatedButtonThemeData(  
      style: ElevatedButton.styleFrom(  
        backgroundColor: Colors.teal,  
        foregroundColor: Colors.white,  
        padding: const EdgeInsets.symmetric(vertical: 14),  
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),  
      ),  
    ),  
  
    inputDecorationTheme: InputDecorationTheme(  
      border: OutlineInputBorder(borderRadius: BorderRadius.circular(10)),  
      filled: true,  
      fillColor: Colors.grey.shade100,  
    ),  
  
    textTheme: const TextTheme(  
      titleLarge: TextStyle(fontSize: 22, fontWeight: FontWeight.bold),  
      bodyMedium: TextStyle(fontSize: 15),  
    ),  
  ),  
  
  darkTheme: ThemeData.dark(useMaterial3: true),  
  themeMode: ThemeMode.system, // ফোনের সেটিং অনুযায়ী  
)
```

থিম থেকে মান ব্যবহার করা

`Theme.of(context).colorScheme.primary` বা `Theme.of(context).textTheme.titleLarge` — এভাবে লিখলে থিম বদলালে পুরো অ্যাপ একসাথে বদলে যাবে। প্রতিটি জায়গায় হাতে রং লিখবেন না।

২. কাস্টম ফন্ট যোগ করা (বাংলা ফন্টসহ)

ফন্ট ফাইল (.ttf) `assets/fonts/` ফোল্ডারে রাখুন, তারপর `pubspec.yaml` -এ:

```
flutter:  
  uses-material-design: true  
  
  assets:  
    - assets/images/  
  
  fonts:  
    - family: HindSiliguri  
      fonts:  
        - asset: assets/fonts/HindSiliguri-Regular.ttf  
        - asset: assets/fonts/HindSiliguri-Bold.ttf  
          weight: 700
```

```
// পুরো অ্যাপে  
ThemeData(fontFamily: 'HindSiliguri')  
  
// শুধু একটি টেক্সটে  
Text('বাংলা লেখা', style: TextStyle(fontFamily: 'HindSiliguri'))
```

বাংলা টেক্সটের জন্য সেরা ফ্রি ফন্ট

Hind Siliguri, Noto Sans Bengali, Baloo Da 2 — সবগুলোই Google Fonts-এ ফ্রি। অথবা `google_fonts` প্যাকেজ ব্যবহার করলে ফাইল ডাউনলোড ছাড়াই কাজ হয়:

```
GoogleFonts.hindSiliguri(fontSize: 18)
```

৩. রেসপন্সিভ ডিজাইন

```
@override
Widget build(BuildContext context) {
  final width = MediaQuery.of(context).size.width;
  final height = MediaQuery.of(context).size.height;

  return Scaffold(
    body: width > 600
      ? const TabletLayout()      // বড় স্ক্রিন
      : const MobileLayout(),     // ছোট স্ক্রিন
  );
}

// LayoutBuilder – প্যারেন্টের মাপ অনুযায়ী
LayoutBuilder(
  builder: (context, constraints) {
    return GridView.count(
      crossAxisCount: constraints.maxWidth > 600 ? 4 : 2,
      children: [...],
    );
  },
)

// নচ ও স্ট্যাটাস বার এড়াতে
SafeArea(child: ...)
```

৪. সহজ অ্যানিমেশন

```
// AnimatedContainer – মান বদলালে নিজে থেকেই মসৃণভাবে বদলায়
AnimatedContainer(
  duration: const Duration(milliseconds: 400),
  curve: Curves.easeInOut,
  width: _isBig ? 200 : 100,
  height: _isBig ? 200 : 100,
  color: _isBig ? Colors.teal : Colors.orange,
)

// AnimatedOpacity – ফেড ইন/আউট
AnimatedOpacity(
  duration: const Duration(milliseconds: 300),
  opacity: _visible ? 1.0 : 0.0,
  child: const Text('আমি ভেসে উঠছি'),
)

// Hero – দুই পেজের মধ্যে ছবি উড়িয়ে নেওয়া
// পেজ ১-এ
Hero(tag: 'product-5', child: Image.network(url))
// পেজ ২-এ একই tag দিন
Hero(tag: 'product-5', child: Image.network(url))

// AnimatedSwitcher – widget বদলের সময় ট্রানজিশন
AnimatedSwitcher(
  duration: const Duration(milliseconds: 300),
  child: _loading
    ? const CircularProgressIndicator(key: ValueKey('load'))
    : const Text('হয়ে গেছে', key: ValueKey('done')),
)
```

৫. অ্যাপের আইকন ও নাম বদলানো

```
# pubspec.yaml-এ যোগ করুন
dev_dependencies:
  flutter_launcher_icons: ^0.13.1

flutter_launcher_icons:
  android: true
  ios: true
  image_path: "assets/icon/icon.png"    # ১০২৪x১০২৪ px

# তারপর টার্মিনালে
flutter pub get
dart run flutter_launcher_icons
```

অ্যাপের নাম: `android/app/src/main/AndroidManifest.xml` ফাইলে `android:label="আপনার অ্যাপের নাম"` বদলান।

আজকের অনুশীলন

আপনার আগের যেকোনো একটি অ্যাপে একটি পূর্ণ থিম যোগ করুন (রং, বাটন স্টাইল, ইনপুট স্টাইল), একটি বাংলা ফন্ট যোগ করুন, একটি ডার্ক মোড টগল বানান, এবং লিস্ট থেকে ডিটেইল পেজে Hero অ্যানিমেশন যোগ করুন।

দিন ১১

API, HTTP ও JSON — ইন্টারনেট থেকে ডেটা আনা

লক্ষ্য: সত্যিকারের সার্ভার থেকে ডেটা এনে অ্যাপে দেখানো।

১. প্যাকেজ যোগ করা

```
# টার্মিনালে (সবচেয়ে সহজ উপায়)
flutter pub add http

# অথবা pubspec.yaml-এ হাতে লিখে
dependencies:
  http: ^1.2.0

# তারপর
flutter pub get
```

২. GET রিকোয়েস্ট — ডেটা আনা

```
import 'dart:convert';
import 'package:http/http.dart' as http;

Future<List<Product>> fetchProducts() async {
  final url = Uri.parse('https://fakestoreapi.com/products');

  final response = await http.get(url);

  if (response.statusCode == 200) {
    final List data = jsonDecode(response.body);
    return data.map((json) => Product.fromJson(json)).toList();
  } else {
    throw Exception('ডেটা আনতে ব্যর্থ (কোড: ${response.statusCode})');
  }
}
```

৩. POST রিকোয়েস্ট — ডেটা পাঠানো

```
Future<void> login(String email, String password) async {
  final response = await http.post(
    Uri.parse('https://api.example.com/login'),
    headers: {
      'Content-Type': 'application/json',
      'Authorization': 'Bearer $token', // দরকার হলে
    },
    body: jsonEncode({
      'email': email,
      'password': password,
    }),
  );

  if (response.statusCode == 200) {
    final data = jsonDecode(response.body);
    print('টোকেন: ${data['token']}');
  } else {
    throw Exception('লগইন ব্যর্থ');
  }
}
```

HTTP স্ট্যাটাস কোডের অর্থ

200 সফল · **201** তৈরি হয়েছে · **400** ভুল রিকোয়েস্ট · **401** অননুমোদিত (টোকেন নেই/ভুল) · **403** নিষিদ্ধ · **404** পাওয়া যায়নি · **500** সার্ভারে সমস্যা।

8. FutureBuilder — API-এর ডেটা স্ক্রিনে দেখানো

```
class ProductListPage extends StatefulWidget {
  const ProductListPage({super.key});
  @override
  State<ProductListPage> createState() => _ProductListPageState();
}

class _ProductListPageState extends State<ProductListPage> {
  late Future<List<Product>> _future;

  @override
  void initState() {
    super.initState();
    _future = fetchProducts(); // ★ build()-এ কল করবেন না!
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('পণ্যসমূহ')),
      body: FutureBuilder<List<Product>>(
        future: _future,
        builder: (context, snapshot) {

          // ১. এখনো লোড হচ্ছে
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }

          // ২. এরর হয়েছে
          if (snapshot.hasError) {
            return Center(
              child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  const Icon(Icons.error_outline, size: 60, color: Colors.red),
                  const SizedBox(height: 12),
                  Text('সমস্যা: ${snapshot.error}'),
                  const SizedBox(height: 12),
                  ElevatedButton(
                    onPressed: () => setState(() => _future = fetchProducts()),
                    child: const Text('আবার চেষ্টা করুন'),
                  ),
                ],
              ),
            );
          }
        },
      ),
    );
  }
}
```

```

// ৩. ডেটা এসেছে কিন্তু খালি
final items = snapshot.data ?? [];
if (items.isEmpty) {
  return const Center(child: Text('কোনো পণ্য নেই'));
}

// ৪. সফল
return ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, i) => ListTile(
    leading: Image.network(items[i].image, width: 50),
    title: Text(items[i].title, maxLines: 1,
      overflow: TextOverflow.ellipsis),
    subtitle: Text('ট ${items[i].price}'),
  ),
);
},
),
);
}
}

```

ইন্টারনেট পারমিশন

Android-এ API কাজ না করলে `android/app/src/main/AndroidManifest.xml` ফাইলে `<manifest>`

ট্যাগের ভিতরে যোগ করুন:

```
<uses-permission android:name="android.permission.INTERNET" />
```

৫. পরিচ্ছন্ন কোড — API Service ক্লাস

```
class ApiService {
    static const String baseUrl = 'https://fakestoreapi.com';

    static Future<List<Product>> getProducts() async {
        try {
            final res = await http
                .get(Uri.parse('$baseUrl/products'))
                .timeout(const Duration(seconds: 15));

            if (res.statusCode == 200) {
                final List data = jsonDecode(res.body);
                return data.map((e) => Product.fromJson(e)).toList();
            }
            throw Exception('সার্ভার এরর: ${res.statusCode}');
        } on SocketException {
            throw Exception('ইন্টারনেট সংযোগ নেই');
        } on TimeoutException {
            throw Exception('সময় শেষ, আবার চেষ্টা করুন');
        }
    }
}
```

প্র্যাকটিসের জন্য ফ্রি API

fakestoreapi.com (প্রোডাক্ট) · jsonplaceholder.typicode.com (পোস্ট, ইউজার) ·
openweathermap.org (আবহাওয়া, ফ্রি কী লাগে) · newsapi.org (খবর) · dog.ceo/dog-api (ছবি)।

আজকের অনুশীলন

১. fakestoreapi.com/products থেকে ডেটা এনে একটি গ্রিডে দেখান।
২. আইটেমে ট্যাপ করলে ডিটেইল পেজে পুরো বর্ণনা দেখান।
৩. লোডিং, এরর ও খালি — তিনটি অবস্থাই ঠিকভাবে হ্যান্ডল করুন।
৪. উপরে সার্চ বক্স দিয়ে ফিল্টার যোগ করুন এবং pull-to-refresh চালু করুন।

দিন ১২

স্টেট ম্যানেজমেন্ট — Provider

লক্ষ্য: একাধিক স্ক্রিনের মধ্যে ডেটা শেয়ার করা।

১. সমস্যাটা কী?

`setState` শুধু একটি widget-এর ভিতরে কাজ করে। কিন্তু ধরুন আপনার শপিং কার্টে একটি পণ্য যোগ করলেন প্রোডাক্ট পেজে — সেটি AppBar-এর কার্ট আইকনের সংখ্যাতেও দেখাতে হবে, আবার কার্ট পেজেও। এভাবে দূরের widget-দের মধ্যে ডেটা শেয়ার করতে **state management** লাগে।

পদ্ধতি	কখন ব্যবহার করবেন
setState	একটি widget-এর নিজস্ব ছোট state
Provider	নতুনদের জন্য সেরা। ছোট-মাঝারি অ্যাপ। এটিই শিখুন।
Riverpod	Provider-এর উন্নত সংস্করণ। পরে শিখলেই হবে।
Bloc	বড় টিম, এন্টারপ্রাইজ অ্যাপ। শেখা কঠিন।
GetX	দ্রুত কাজ, কিন্তু অনেক খারাপ অভ্যাস তৈরি করে। এড়িয়ে চলুন।

২. Provider সেটআপ

```
flutter pub add provider
```

ধাপ ১ — ChangeNotifier ক্লাস বানান

```
import 'package:flutter/material.dart';

class CartProvider extends ChangeNotifier {
  final List<Product> _items = [];

  // বাইরে থেকে শুধু পড়া যাবে, বদলানো যাবে না
  List<Product> get items => List.unmodifiable(_items);
  int get count => _items.length;
  double get total => _items.fold(0, (sum, p) => sum + p.price);

  void add(Product p) {
    _items.add(p);
    notifyListeners(); // ★ সব শ্রোতাকে জানিয়ে দাও
  }

  void remove(Product p) {
    _items.remove(p);
    notifyListeners();
  }

  void clear() {
    _items.clear();
    notifyListeners();
  }
}
```

ধাপ ২ — অ্যাপের উপরে Provider বসান

```
void main() {
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => CartProvider()),
        ChangeNotifierProvider(create: (_) => ThemeProvider()),
      ],
      child: const MyApp(),
    ),
  );
}
```

ধাপ ৩ — যেকোনো widget থেকে ব্যবহার

```
// ক) ডেটা পড়া + পরিবর্তনে rebuild হওয়া
final cart = context.watch<CartProvider>();
Text('কার্টে ${cart.count} টি পণ্য');

// খ) শুধু ফাংশন কল করা (rebuild হবে না – onPressed-এ এটি ব্যবহার করুন)
context.read<CartProvider>().add(product);

// গ) Consumer – শুধু নির্দিষ্ট অংশ rebuild করতে
Consumer<CartProvider>(
  builder: (context, cart, child) => Badge(
    label: Text('${cart.count}'),
    child: const Icon(Icons.shopping_cart),
  ),
)

// ঘ) Selector – শুধু একটি মান বদলালে rebuild (সবচেয়ে দক্ষ)
Selector<CartProvider, int>(
  selector: (_, cart) => cart.count,
  builder: (_, count, __) => Text('$count'),
)
```

সবচেয়ে সাধারণ Provider ভুল

১. `notifyListeners()` ডাকতে ভুলে যাওয়া → UI বদলায় না।
২. `onPressed` -এ `watch` ব্যবহার → অপ্রয়োজনীয় rebuild। সেখানে `read` ব্যবহার করুন।
৩. `build()` -এর বাইরে `watch` ব্যবহার → এরর।
৪. Provider-কে যেই widget ব্যবহার করবে তার **উপরে** না বসানো → "Could not find the correct Provider" এরর।

৩. পূর্ণ উদাহরণ — ডার্ক মোড টগল

```
class ThemeProvider extends ChangeNotifier {
  bool _isDark = false;
  bool get isDark => _isDark;

  void toggle() {
    _isDark = !_isDark;
    notifyListeners();
  }
}

// main.dart-এ
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    final theme = context.watch<ThemeProvider>();
    return MaterialApp(
      themeMode: theme.isDark ? ThemeMode.dark : ThemeMode.light,
      theme: ThemeData.light(useMaterial3: true),
      darkTheme: ThemeData.dark(useMaterial3: true),
      home: const HomePage(),
    );
  }
}

// যেকোনো পেজে সুইচ
Switch(
  value: context.watch<ThemeProvider>().isDark,
  onChanged: (_) => context.read<ThemeProvider>().toggle(),
)
```

আজকের অনুশীলন — শপিং কার্ট

দিন ১১-এর প্রোডাক্ট অ্যাপে CartProvider যোগ করুন। প্রতিটি প্রোডাক্টে "কার্টে যোগ করুন" বাটন থাকবে, AppBar-এ কার্ট আইকনের উপর সংখ্যা দেখাবে, এবং কার্ট পেজে সব পণ্য ও মোট দাম দেখাবে। কার্ট থেকে পণ্য মুছে ফেলার ব্যবস্থাও রাখুন।

দিন ১৩

লোকাল স্টোরেজ ও ডেটাবেজ

লক্ষ্য: অ্যাপ বন্ধ করলেও ডেটা যেন থেকে যায়।

১. shared_preferences — ছোট ডেটা সেভ করা

লগইন টোকেন, ইউজারের নাম, ডার্ক মোড চালু কিনা — এসব ছোট ডেটার জন্য।

```
flutter pub add shared_preferences
```

```
import 'package:shared_preferences/shared_preferences.dart';

// সেভ করা
Future<void> saveData() async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('username', 'রহিম');
  await prefs.setInt('age', 22);
  await prefs.setBool('isDarkMode', true);
  await prefs.setStringList('hobbies', ['পড়া', 'খেলা']);
}

// পড়া
Future<void> loadData() async {
  final prefs = await SharedPreferences.getInstance();
  final name = prefs.getString('username') ?? 'অতিথি';
  final isDark = prefs.getBool('isDarkMode') ?? false;
  print('$name, ডার্ক: $isDark');
}

// মুছে ফেলা (লগআউটের সময়)
Future<void> logout() async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.remove('token'); // একটি
  await prefs.clear();         // সব
}
```

২. ব্যবহারিক — লগইন অবস্থা মনে রাখা

```
class SplashPage extends StatefulWidget {
  const SplashPage({super.key});
  @override
  State<SplashPage> createState() => _SplashPageState();
}

class _SplashPageState extends State<SplashPage> {
  @override
  void initState() {
    super.initState();
    _checkLogin();
  }

  Future<void> _checkLogin() async {
    await Future.delayed(const Duration(seconds: 2)); // স্প্যাশ দেখানো
    final prefs = await SharedPreferences.getInstance();
    final token = prefs.getString('token');

    if (!mounted) return; // ★ async এর পরে খুব গুরুত্বপূর্ণ

    Navigator.pushReplacement(
      context,
      MaterialPageRoute(
        builder: (_) => token == null ? const LoginPage() : const HomePage(),
      ),
    );
  }

  @override
  Widget build(BuildContext context) => const Scaffold(
    body: Center(child: FlutterLogo(size: 100)),
  );
}
```

"Don't use BuildContext across async gaps"

`await` -এর পরে `context` ব্যবহার করলে এই সতর্কতা আসে — কারণ ততক্ষণে widget-টি হয়তো আর স্ক্রিনে নেই। সমাধান: আগে `if (!mounted) return;` লিখুন।

৩. sqflite — পূর্ণাঙ্গ লোকাল ডেটাবেজ

অনেক ডেটা, সার্চ ও সম্পর্কযুক্ত টেবিল দরকার হলে SQLite ব্যবহার করুন।

```
flutter pub add sqflite path
```

```

import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

class DBHelper {
  static Database? _db;

  static Future<Database> get database async {
    if (_db != null) return _db!;
    final path = join(await getDatabasesPath(), 'notes.db');

    _db = await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''
          CREATE TABLE notes(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            body TEXT,
            createdAt TEXT
          )
        ''');
      },
    );
    return _db!;
  }

  // Create
  static Future<int> insertNote(Map<String, dynamic> note) async {
    final db = await database;
    return db.insert('notes', note);
  }

  // Read
  static Future<List<Map<String, dynamic>>> getNotes() async {
    final db = await database;
    return db.query('notes', orderBy: 'id DESC');
  }

  // Update
  static Future<int> updateNote(int id, Map<String, dynamic> note) async {
    final db = await database;
    return db.update('notes', note, where: 'id = ?', whereArgs: [id]);
  }

  // Delete
  static Future<int> deleteNote(int id) async {
    final db = await database;
    return db.delete('notes', where: 'id = ?', whereArgs: [id]);
  }

```

```
}

// Search
static Future<List<Map<String, dynamic>>> search(String q) async {
  final db = await database;
  return db.query('notes', where: 'title LIKE ?', whereArgs: ['%$q%']);
}
}
```

কোনটি কখন?

shared_preferences — কয়েকটি সেটিং বা টোকেন। **sqflite** — শত শত রেকর্ড, সার্চ, সম্পর্ক। **hive** — দ্রুত ও সহজ NoSQL বিকল্প (SQL না জানলে ভালো)। **flutter_secure_storage** — পাসওয়ার্ড ও টোকেনের মতো সংবেদনশীল ডেটা এনক্রিপ্ট করে রাখতে।

আজকের অনুশীলন — নোট অ্যাপ

sqflite দিয়ে একটি সম্পূর্ণ নোট অ্যাপ বানান: নোট যোগ, তালিকা দেখা, সম্পাদনা, মুছে ফেলা ও সার্চ। সাথে shared_preferences দিয়ে ডার্ক মোডের সেটিং সেভ করে রাখুন যাতে অ্যাপ বন্ধ করে খুললেও তা মনে থাকে।

Firestore, প্যাকেজ, পারমিশন ও ডিভাইস ফিচার

লক্ষ্য: রিয়েল-ওয়ার্ল্ড অ্যাপের ফিচারগুলো যোগ করা।

১. দরকারি প্যাকেজের তালিকা

প্যাকেজ	কাজ
http / dio	API কল (dio-তে ইন্টারসেপ্টর ও ফাইল আপলোড সহজ)
provider / flutter_riverpod	স্টেট ম্যানেজমেন্ট
shared_preferences	ছোট ডেটা সেভ
sqlite / hive	লোকাল ডেটাবেজ
image_picker	ক্যামেরা বা গ্যালারি থেকে ছবি
geolocator	GPS লোকেশন
permission_handler	রানটাইম পারমিশন চাওয়া
cached_network_image	ছবি ক্যাশ করে দ্রুত লোড
intl	তারিখ ও সংখ্যা ফরম্যাট, ভাষা
url_launcher	ফোন কল, ইমেইল, ব্রাউজার খোলা
flutter_local_notifications	লোকাল নোটিফিকেশন
google_fonts	হাজারো ফন্ট সরাসরি ব্যবহার
connectivity_plus	ইন্টারনেট আছে কিনা যাচাই
flutter_svg	SVG ছবি দেখানো

২. পারমিশন চাওয়া

Android-এ `android/app/src/main/AndroidManifest.xml` -এ:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

```
import 'package:permission_handler/permission_handler.dart';

Future<void> askCamera() async {
  final status = await Permission.camera.request();

  if (status.isGranted) {
    // অনুমতি পেয়েছি
  } else if (status.isPermanentlyDenied) {
    await openAppSettings(); // সেটিংসে পাঠিয়ে দিন
  }
}
```

৩. ছবি তোলা / বাছাই

```
import 'dart:io';
import 'package:image_picker/image_picker.dart';

File? _image;

Future<void> pickImage(ImageSource source) async {
  final picked = await ImagePicker().pickImage(
    source: source, // ImageSource.camera বা .gallery
    imageQuality: 70, // সাইজ কমানোর জন্য
    maxWidth: 1080,
  );
  if (picked != null) {
    setState(() => _image = File(picked.path));
  }
}

// দেখানো
_image == null
  ? const Icon(Icons.person, size: 80)
  : ClipOval(child: Image.file(_image!, width: 120, height: 120,
    fit: BoxFit.cover))
```

৪. লোকেশন নেওয়া

```
import 'package:geolocator/geolocator.dart';

Future<Position> getLocation() async {
  // GPS চালু আছে কিনা
  if (!await Geolocator.isLocationServiceEnabled()) {
    throw Exception('অনুগ্রহ করে GPS চালু করুন');
  }

  // পারমিশন
  var permission = await Geolocator.checkPermission();
  if (permission == LocationPermission.denied) {
    permission = await Geolocator.requestPermission();
  }
  if (permission == LocationPermission.deniedForever) {
    throw Exception('লোকেশন পারমিশন স্থায়ীভাবে বন্ধ');
  }

  return await Geolocator.getCurrentPosition(
    desiredAccuracy: LocationAccuracy.high,
  );
}
```

৫. Firebase — ব্যাকএন্ড ছাড়াই ব্যাকএন্ড

```
# ধাপ ১: FlutterFire CLI ইনস্টল
dart pub global activate flutterfire_cli

# ধাপ ২: প্রজেক্টে কনফিগার (console.firebase.google.com-এ প্রজেক্ট বানিয়ে নিন)
flutterfire configure

# ধাপ ৩: প্যাকেজ
flutter pub add firebase_core firebase_auth cloud_firestore
```

```
// main.dart
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MyApp());
}

// রেজিস্ট্রেশন
await FirebaseAuth.instance.createUserWithEmailAndPassword(
  email: email, password: password,
);

// লগইন
await FirebaseAuth.instance.signInWithEmailAndPassword(
  email: email, password: password,
);

// লগআউট
await FirebaseAuth.instance.signOut();

// Firestore-এ ডেটা লেখা
await FirebaseFirestore.instance.collection('notes').add({
  'title': 'আমার নোট',
  'createdAt': FieldValue.serverTimestamp(),
});

// রিয়েলটাইম ডেটা পড়া (StreamBuilder দিয়ে)
StreamBuilder<QuerySnapshot>(
  stream: FirebaseFirestore.instance.collection('notes').snapshots(),
  builder: (context, snapshot) {
    if (!snapshot.hasData) return const CircularProgressIndicator();
    final docs = snapshot.data!.docs;
    return ListView.builder(
      itemCount: docs.length,
      itemBuilder: (c, i) => ListTile(title: Text(docs[i]['title'])),
    );
  },
)
```

FutureBuilder বনাম StreamBuilder

FutureBuilder — একবার ডেটা এনে দেখায় (API কলের জন্য)।

StreamBuilder — ডেটা বদলালেই সাথে সাথে স্ক্রিন আপডেট হয় (Firestore, চ্যাট, লাইভ ডেটার জন্য)।

আজকের অনুশীলন

Firebase Auth দিয়ে একটি লগইন/রেজিস্ট্রেশন সিস্টেম বানান। লগইনের পর প্রোফাইল পেজে image_picker দিয়ে ছবি বদলানোর সুবিধা রাখুন এবং Firestore-এ নোট সেভ ও দেখানোর ব্যবস্থা করুন।

দিন ১৫

ফাইনাল প্রজেক্ট, ডিবাগিং ও APK রিলিজ

লক্ষ্য: সব শেখা একসাথে জুড়ে একটি পূর্ণ অ্যাপ বানানো ও প্রকাশ করা।

১. প্রজেক্ট ফোল্ডার সাজানোর সঠিক নিয়ম

```
lib/  
├─ main.dart  
├─ models/           // ডেটা ক্লাস – product.dart, user.dart  
├─ screens/           // প্রতিটি পেজ – home_screen.dart, login_screen.dart  
├─ widgets/           // পুনঃব্যবহারযোগ্য অংশ – product_card.dart  
├─ services/           // API ও ডেটাবেজ – api_service.dart, db_helper.dart  
├─ providers/         // স্টেট – cart_provider.dart  
└─ utils/             // সহায়ক – constants.dart, validators.dart
```

২. ফাইনাল প্রজেক্ট — যেকোনো একটি বেছে নিন

প্রজেক্ট	যেসব ফিচার থাকবে
নোট অ্যাপ	লগইন, নোট যোগ/সম্পাদনা/মুছে ফেলা, সার্চ, ডার্ক মোড, sqflite
ই-কমার্স	API থেকে পণ্য, ক্যাটাগরি, ডিটেইল পেজ, কার্ট (Provider), সার্চ
আবহাওয়া অ্যাপ	GPS লোকেশন, OpenWeather API, ৭ দিনের পূর্বাভাস, সুন্দর UI
খরচের হিসাব	আয়/ব্যয় যোগ, ক্যাটাগরি, মাসিক চার্ট, লোকাল ডেটাবেজ
খবরের অ্যাপ	NewsAPI, ক্যাটাগরি ট্যাব, বুকমার্ক, শেয়ার, ওয়েবভিউ

প্রজেক্ট বানানোর ধাপ

১. কাগজে স্ক্রিনগুলো ঐকে নিন। ২. মডেল ক্লাস লিখুন। ৩. স্ট্যাটিক (ভুয়া) ডেটা দিয়ে UI বানান। ৪. তারপর API/ডেটাবেজ যুক্ত করুন। ৫. সবশেষে থিম ও অ্যানিমেশন দিয়ে সাজান।
কখনোই একসাথে সব করতে যাবেন না — একটি স্ক্রিন শেষ করে পরেরটি ধরুন।

৩. ডিবাগিং — সমস্যা খুঁজে বের করা

```
// প্রিন্ট করে দেখা
print('মান: $value');
debugPrint('লম্বা লেখার জন্য এটি ভালো');

// শর্ত না মিললে থামিয়ে দেওয়া
assert(items.isNotEmpty, 'লিস্ট খালি হওয়ার কথা নয়');

// পুরো widget tree দেখা
debugDumpApp();
```

```
# দরকারি কমান্ড
flutter clean           # অদ্ভুত এরর হলে প্রথমেই এটি চালান
flutter pub get
flutter analyze         # কোডের ভুল ও সতর্কতা দেখায়
dart format .           # কোড সুন্দর করে সাজায়
flutter doctor -v
```

Flutter DevTools

অ্যাপ চলা অবস্থায় VS Code-এ Ctrl+Shift+P → "Flutter: Open DevTools"। এখানে **Widget Inspector** দিয়ে লেআউট সমস্যা ধরা যায়, **Performance** ট্যাবে অ্যাপ ধীর হওয়ার কারণ দেখা যায়।

৪. রিলিজ APK তৈরি

```
# দ্রুত পরীক্ষার জন্য – সবার ফোনে চলবে
flutter build apk --release

# ছোট সাইজের আলাদা APK (সুপারিশকৃত)
flutter build apk --split-per-abi

# Play Store-এ আপলোডের জন্য
flutter build appbundle --release

# iOS (শুধু macOS-এ)
flutter build ios --release
```

APK ফাইলটি পাবেন এখানে: `build/app/outputs/flutter-apk/app-release.apk`

৫. Play Store-এর জন্য অ্যাপ সাইন করা

ধাপ ১: কী তৈরি করুন

```
keytool -genkey -v -keystore ~/upload-keystore.jks -keyalg RSA \
-keysize 2048 -validity 10000 -alias upload
```

ধাপ ২: android/key.properties ফাইল বানান (এটি git-এ দেবেন না!)

```
storePassword=আপনার_পাসওয়ার্ড
keyPassword=আপনার_পাসওয়ার্ড
keyAlias=upload
storeFile=C:/Users/আপনার_নাম/upload-keystore.jks
```

তারপর `android/app/build.gradle` -এ `signingConfigs` যোগ করে `buildTypes { release { signingConfig signingConfigs.release } }` সেট করুন।

keystore ফাইল হারাবেন না!

এই ফাইল ও পাসওয়ার্ড হারালে আপনি আর কখনোই Play Store-এ সেই অ্যাপের আপডেট দিতে পারবেন না। ব্যাকআপ রাখুন, কিন্তু কখনোই GitHub-এ পুশ করবেন না।

৬. এরপর কী শিখবেন?

1. **Riverpod বা Bloc** — বড় অ্যাপের স্টেট ম্যানেজমেন্ট।
2. **Clean Architecture** — repository pattern, dependency injection (get_it)।
3. **Testing** — unit test, widget test, integration test।
4. **go_router** — আধুনিক ও শক্তিশালী নেভিগেশন, deep linking।
5. **Custom Painter ও Animation** — নিজের মতো গ্রাফিক্স ও জটিল অ্যানিমেশন।
6. **CI/CD** — GitHub Actions বা Codemagic দিয়ে অটো বিল্ড ও রিলিজ।
7. **Platform Channel** — Kotlin/Swift-এর নেটিভ কোড Flutter থেকে ডাকা।

চূড়ান্ত কাজ

১. আপনার প্রজেক্টটি GitHub-এ পুশ করুন এবং একটি ভালো README লিখুন (স্ক্রিনশটসহ)।
২. রিলিজ APK বানিয়ে বন্ধুদের ফোনে ইনস্টল করে দেখান।
৩. LinkedIn-এ প্রজেক্টটি নিয়ে একটি পোস্ট দিন — এটি চাকরি পেতে সাহায্য করে।

পরিশিষ্ট ক — Widget চিটশিট

লেআউট Widget

Widget	কাজ
Container	মাপ, রং, বর্ডার, প্যাডিং, ছায়া — সবকিছু একসাথে
Row / Column	পাশাপাশি / উপর-নিচে সাজানো
Stack / Positioned	একটির উপর আরেকটি বসানো
Expanded / Flexible	বাকি জায়গা ভাগ করে নেওয়া
SizedBox	নির্দিষ্ট মাপ বা ফাঁকা জায়গা
Padding	চারপাশে ফাঁকা জায়গা
Center / Align	মাঝখানে বা নির্দিষ্ট দিকে বসানো
Wrap	জায়গা না থাকলে পরের লাইনে নামিয়ে দেয়
SafeArea	নচ ও স্ট্যাটাস বার এড়ানো
SingleChildScrollView	ছোট কনটেন্ট স্ক্রলযোগ্য করা
ListView / GridView	লম্বা লিস্ট বা গ্রিড
AspectRatio	প্রস্থ ও উচ্চতার অনুপাত ঠিক রাখা
FittedBox	ভিতরের জিনিস মাপমতো ছোট-বড় করা

দৃশ্যমান ও ইনপুট Widget

Widget	কাজ
Text / RichText	লেখা (RichText-এ এক লাইনে একাধিক স্টাইল)
Image / Icon	ছবি ও আইকন
Card / ListTile	রেডিমেড কার্ড ও লিস্ট আইটেম
CircleAvatar	গোল প্রোফাইল ছবি
Divider	বিভাজক রেখা
Chip / Badge	ছোট ট্যাগ ও সংখ্যা ব্যাজ
TextField / TextFormField	লেখা ইনপুট
Checkbox / Switch / Radio	বাছাই করার ইনপুট
DropDownButton / Slider	তালিকা ও স্লাইডার
GestureDetector / InkWell	যেকোনো widget-এ ট্যাপ যোগ করা
CircularProgressIndicator	লোডিং ঘূর্ণি
FutureBuilder / StreamBuilder	async ডেটা থেকে UI বানানো
ClipRRect / ClipOval	কোণা গোল করা বা গোল কেটে নেওয়া

দরকারি টার্মিনাল কমান্ড

কমান্ড	কাজ
flutter create নাম	নতুন প্রজেক্ট
flutter run	অ্যাপ চালানো
flutter run -d chrome	ব্রাউজারে চালানো
flutter devices	যুক্ত ডিভাইসের তালিকা
flutter pub add প্যাকেজ	প্যাকেজ যোগ
flutter pub get	প্যাকেজ ডাউনলোড
flutter pub upgrade	প্যাকেজ আপডেট
flutter clean	ক্যাশ পরিষ্কার (এরর হলে চালান)
flutter analyze	কোডের ভুল খোঁজা
dart format .	কোড সাজানো
flutter build apk --release	রিলিজ APK
flutter upgrade	Flutter নিজে আপডেট

পরিশিষ্ট খ — সাধারণ এরর ও সমাধান

এরর	সমাধান
RenderFlex overflowed by X pixels	Row-এর সন্তানকে Expanded দিয়ে মুড়ুন, অথবা Column-কে SingleChildScrollView -এ রাখুন।
Vertical viewport was given unbounded height	ListView-কে Expanded -এ রাখুন, অথবা shrinkWrap: true ও NeverScrollableScrollPhysics() দিন।
setState() called after dispose()	setState-এর আগে if (!mounted) return; লিখুন।
Null check operator used on a null value	! -এর বদলে ?? বা null চেক ব্যবহার করুন।
Could not find the correct Provider	Provider-কে ব্যবহারকারী widget-এর উপরে (সাধারণত main.dart-এ) বসান।
Unable to load asset	pubspec.yaml-এ পথ ও ইনডেন্টেশন যাচাই করুন → flutter pub get → অ্যাপ পুরো রিস্টার্ট (hot reload নয়)।
Gradle task assembleDebug failed	flutter clean → flutter pub get । না হলে android/app/build.gradle -এ minSdkVersion বাড়িয়ে ২১+ করুন।
SocketException / ClientException	ইন্টারনেট পারমিশন যোগ করুন; ইমুলেটরে localhost -এর বদলে 10.0.2.2 ব্যবহার করুন।
A RenderFlex was given unbounded width	Row-এর ভিতরে Row/TextField থাকলে Expanded দিয়ে মুড়ুন।
Instance of 'Future<String>' প্রিন্ট হচ্ছে	await লিখতে ভুলে গেছেন।
Waiting for another flutter command to release the startup lock	Flutter SDK ফোল্ডারের bin/cache/lockfile মুছে ফেলুন।
BoxDecoration cannot be used with color	Container-এ color ও decoration একসাথে নয় — রংটি BoxDecoration-এর ভিতরে দিন।

পরিশিষ্ট গ — শেখার রিসোর্স

অফিসিয়াল (সবচেয়ে নির্ভরযোগ্য)

- **docs.flutter.dev** — অফিসিয়াল ডকুমেন্টেশন। যেকোনো প্রশ্নের উত্তর এখানেই আছে।
- **pub.dev** — সব প্যাকেজের ভাণ্ডার। প্যাকেজ বাছার আগে Likes ও শেষ আপডেটের তারিখ দেখুন।
- **api.flutter.dev** — প্রতিটি widget-এর সব প্যারামিটারের বিস্তারিত।
- **YouTube: Flutter** — "Widget of the Week" সিরিজটি অসাধারণ, প্রতিটি ভিডিও ১ মিনিটের।
- **dartpad.dev** — ব্রাউজারেই Dart ও Flutter কোড চালানো।

বাংলায় শেখার জায়গা

- YouTube-এ সার্চ করুন: "**Flutter Bangla Tutorial**", "**Dart Bangla**"।
- Facebook গ্রুপ: **Flutter Bangladesh, Bangladesh Flutter Developers** — প্রশ্ন করলে সাহায্য পাবেন।
- **Stack Overflow** — এরর মেসেজ হুবহু কপি করে সার্চ দিন, ৯০% সমাধান পেয়ে যাবেন।

শেখার সময় সবচেয়ে জরুরি ৭টি অভ্যাস

1. **প্রতিদিন কোড লিখুন** — ৩০ মিনিট হলেও। ৭ দিন পর ৭ ঘণ্টার চেয়ে প্রতিদিন ১ ঘণ্টা ভালো।
2. **এরর দেখে ভয় পাবেন না** — এরর মেসেজ পড়ুন, সেখানেই সমাধান লেখা থাকে।
3. **টিউটোরিয়াল দেখে থেমে যাবেন না** — ভিডিও শেষে সেটি বন্ধ করে নিজে আবার বানান।
4. **ছোট প্রজেক্ট শেষ করুন** — ১০টি অসম্পূর্ণ প্রজেক্টের চেয়ে ২টি সম্পূর্ণ প্রজেক্ট মূল্যবান।
5. **কোড পড়ুন** — GitHub-এ ওপেন সোর্স Flutter অ্যাপ খুলে দেখুন কীভাবে লেখা হয়েছে।
6. **প্রশ্ন করুন** — আটকে গেলে ২০ মিনিটের বেশি একা বসে থাকবেন না।
7. **নিজের কাজ দেখান** — GitHub ও LinkedIn-এ প্রজেক্ট শেয়ার করুন।

শেষ কথা

১৫ দিন পর আপনি যা পারবেন তা অনেকেই ৩ মাসেও পারে না — শর্ত একটাই, **প্রতিদিন কোড লিখতে হবে**। প্রথম সপ্তাহে বারবার আটকে যাবেন, মনে হবে "আমার দ্বারা হবে না"। এটা একদম স্বাভাবিক — প্রত্যেক ডেভেলপার এই পথ দিয়ে গেছে। লেগে থাকুন। দশম দিনের পর হঠাৎ সবকিছু সহজ মনে হতে শুরু করবে।

শুভকামনা রইল! 🚀